

Notes on Neural Networks: part 3 – Deep NN

Alessio Micheli

micheli@di.unipi.it

2025



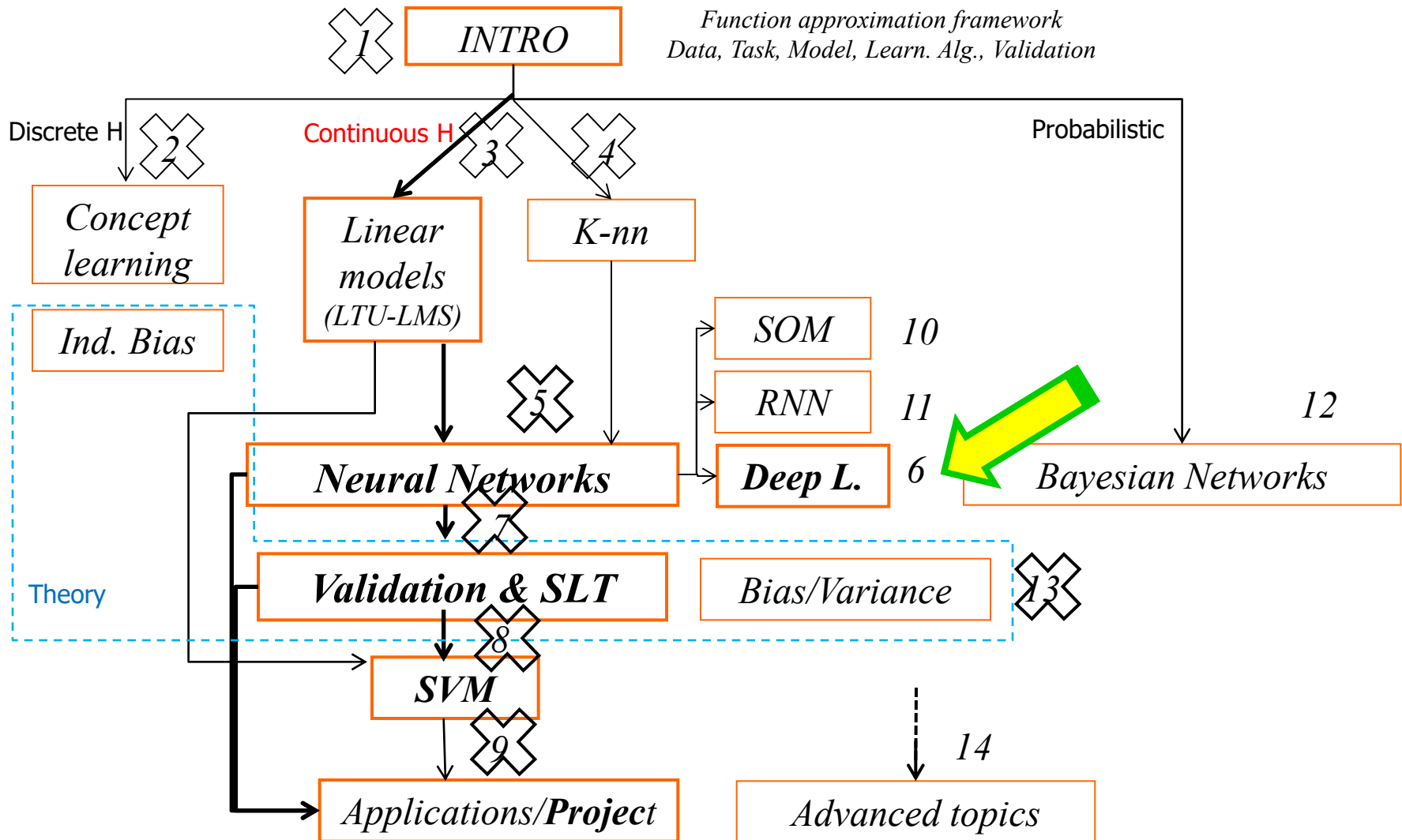
Dipartimento di Informatica
Università di Pisa - Italy

ML Course structure

Where we go



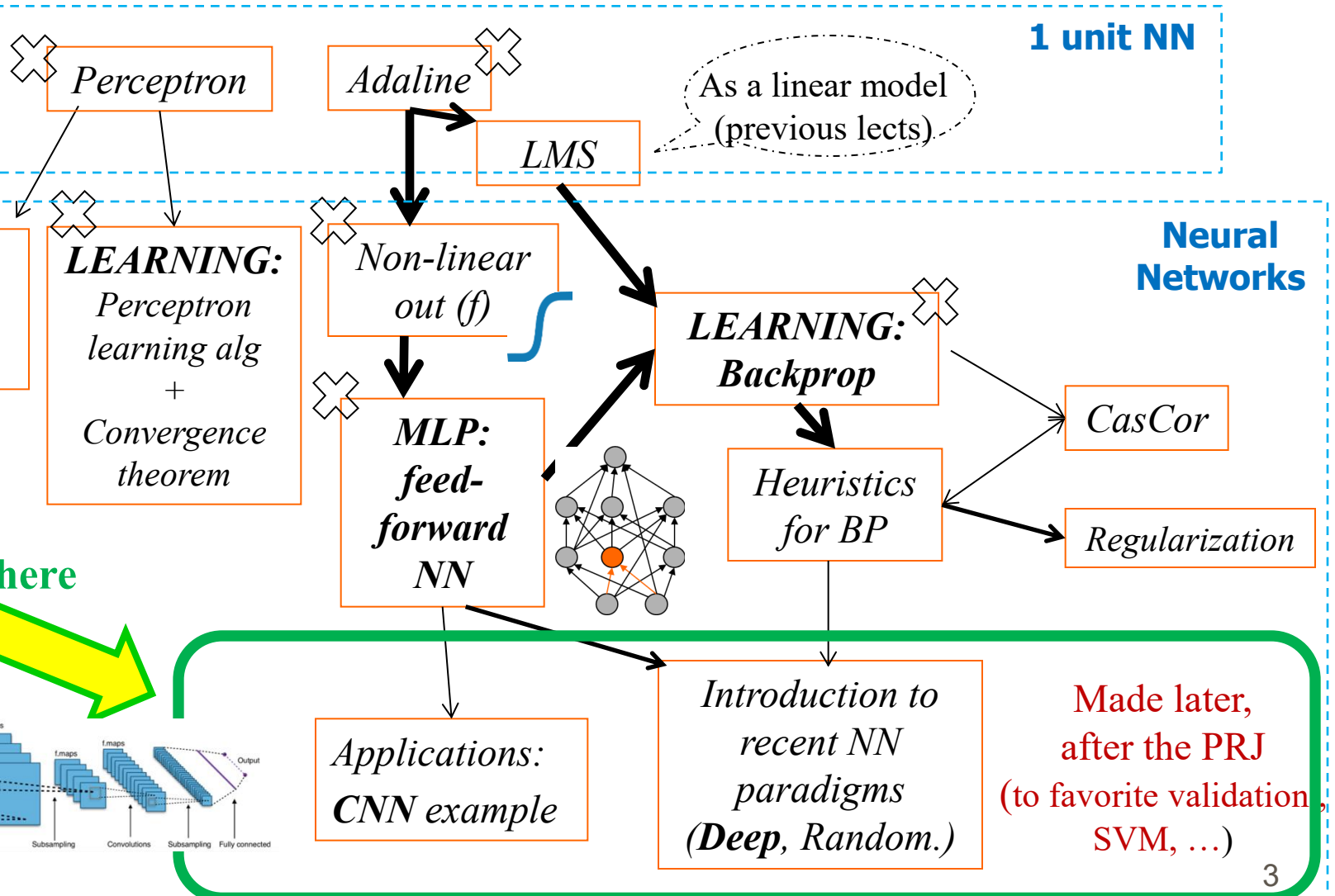
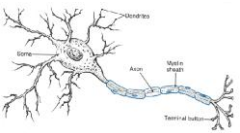
Dip. Informatica
University of Pisa



Course structure: zoom in the NNs box



Dip. Informatica
University of Pisa



Initial messages: Aims in the ML course



Dip. Informatica
University of Pisa

1. Aims for DL in this course and ... overlapping with other courses?
 - (also valid for CNN, RNN, etc)
 - In this course the perspective is to see different NN **architectures** (which is a topic part of the ML course)
 - AI curriculum: In the following courses you will study details, variants aimed at applications to pattern recognition, NLP, etc.
 - Moreover, we need this introduction to recent paradigms for the NN models (we will see that many aspects and techniques are useful for the basic setting)
 - And, also, not all the ML attendants can follow the other courses.
2. Send to me questions for the next Q&A! (tag: [ML-25-Q])

Other architectures (guide slide)



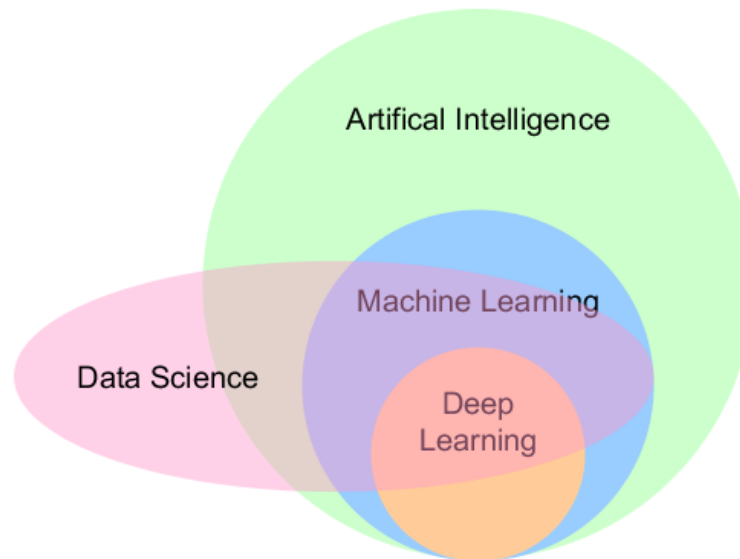
Dip. Informatica
University of Pisa

Introduction to recent NN paradigms

- Other architectures.....

- Deep learning
- Random weights NN

AKA: *Deep* learning
and *extreme* learning ;-)



*Not all the story
but nice view at a
glance*

Deep Learning- guideline

- **Basic (or rapid introduction)**
- Insights (do we need many layers? + Representation learning and distributed representation)
- Techniques (some)
- Applications → further courses in AI (PR, NLP, ...)

Deep Learning quotation



Dip. Informatica
University of Pisa

REF for overview: DL book Chapter 1

- *"Deep learning allows computational models that are composed of multiple processing layers to learn representations of data with multiple levels of abstraction. These methods have dramatically improved the state-of-the-art in speech recognition, visual object recognition, object detection and many other domains"*

LeCun, Y., Bengio, Y., & Hinton, G. (May 2015). *Deep learning*. Nature, 521(7553), 436-444.

Deep Learning: *short history*



Dip. Informatica
University of Pisa

- Starting from 2010 there was an impact of **deep learning** in industry: The use of deep architectures provided new performance results and new insights for new NN approaches
 - Large data sets are nowadays available (e.g. millions of images)
 - Powerful computing system (HPC) are nowadays available (enabling factor, e.g. GPU)
 - New ideas to easy the training (see later)
- ➔ Learning in deep architectures it is no longer “unfeasible”
- ❖ New successful state of the art results for complex task and competitions in *speech recognition* , *machine translation** and *image recognition* tasks attracted many practitioners and research to deep learning approaches.
- ❖ (In the past it was expected an improvement changing the scale of models and data, but not necessarily such *jump* of performance)
- ❖ All the big CS companies are basing their RD strategies on such approaches
 - * e.g. *google (neural) translator since summer 2016, DeepL, etc.*

Deep Learning framework

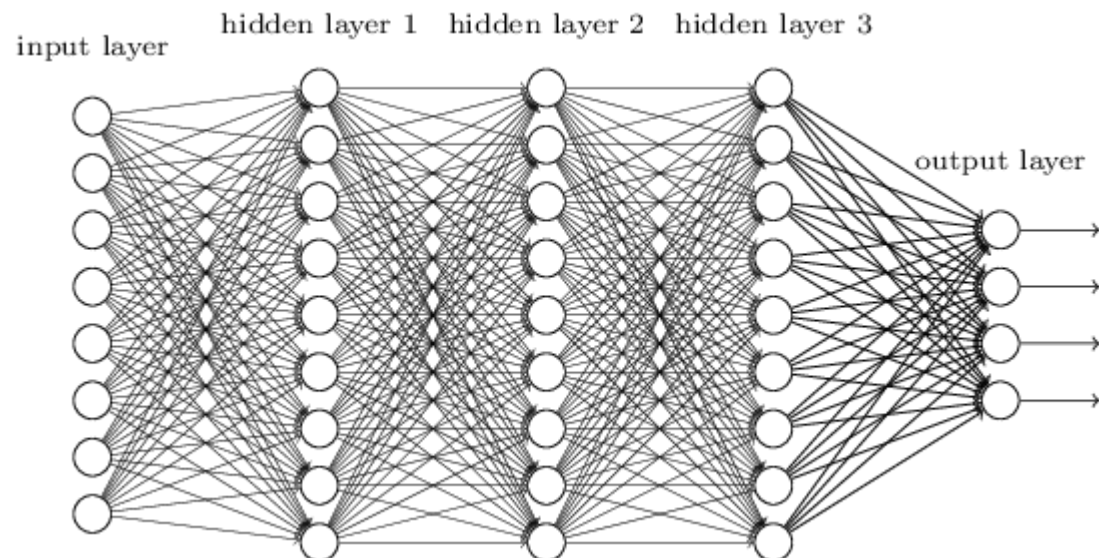


Dip. Informatica
University of Pisa

- It is a general framework including different models, such are:
 - deep Neural Networks (aka a MLP with many layers)
 - *convolutional* NNs → successful for computer vision
 - deep belief networks and generative approaches
 - recurrent and recursive NNs → *active research in Unipi-CIML*
 - ...
- By contrast: “Shallow models” (e.g. a NN with few layers, say less than 3)
- And give to us the opportunity to focus on some relevant concepts, not discussed enough up to now e.g.
 - Representation learning
 - Distributed representation
 - Other techniques

How to build it?

- Many approaches to implement deep learning
- We will introduce approaches well rooted in your background through this course
- Basically by building a MLP with many layers and e.g. pre-training techniques



A more general view

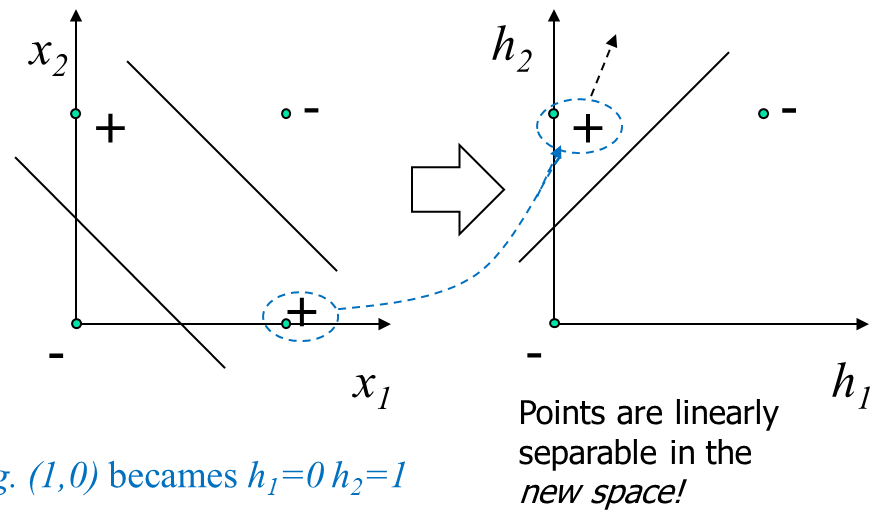
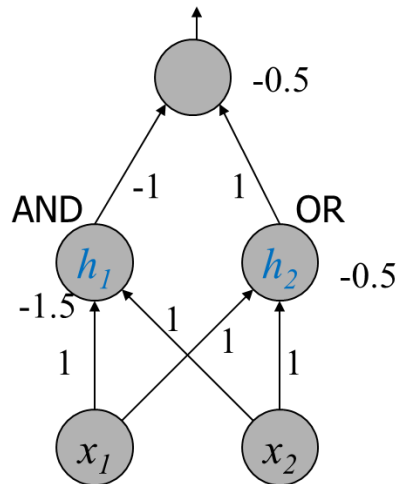
Common aspects among different approaches:

- Multiple layers of nonlinear processing units
- Supervised or unsupervised learning of feature representations in each layer, with the layers forming a hierarchy from low-level to high-level features (different levels of abstraction).
- Hence, hierarchical sparse/distributed representation of the input data
- (differences according to the specific models, e.g. max pool operators for CNN, energy based training for RBM - Restricted Boltzmann Machines , etc.)

REPETITA: XOR by a Two Layers Network

$$x_1 \oplus x_2 = x_1 \cdot \overline{x_2} + \overline{x_1} \cdot x_2 \quad \text{let :} \quad \begin{aligned} h_1 &= x_1 \cdot x_2 \\ h_2 &= x_1 + x_2 \end{aligned} \quad (\text{dot} = \mathbf{and}) \quad (+ = \mathbf{or})$$

then we have: $x_1 \oplus x_2 = \overline{h_1} \cdot h_2$



- Concept of internal **(re)representation** of input variables via internal (hidden) units,
- developing **high level (hidden) features**, is a key factor in NN
- The representation in the hidden layer makes easier the task to the output layer (last unit)
- Such **composition** of sub-/intermediate operations to perform a complex task can be extended through many layers of **abstraction**
- Learning internal distributed representation of the data

Conceptual ingredients: hierarchical abstraction

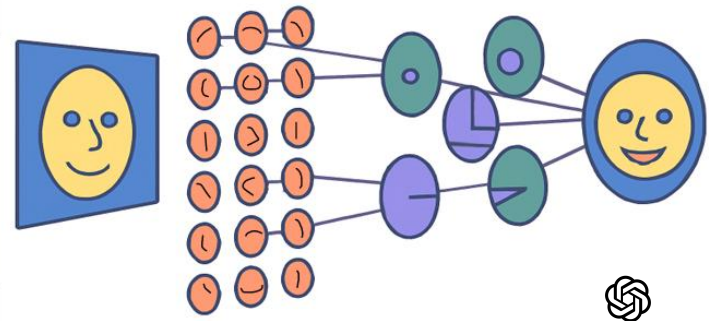
- Unsupervised or semi-supervised feature learning and *hierarchical feature* extraction instead of features engineering (extracted by experience, e.g. the texture from images).

Move to the concept of “**learning representations of data**”

- Increase level of **abstraction through different layers**

Example: An observation (e.g., an image) can be represented in many ways such as a vector of intensity values per pixel, or in a more abstract way as a set of edges, regions of particular shape, etc..

- Edges form motifs, motifs assemble into parts, parts form objects.... *Many levels.*



- Performs best when input information has some form of structure, e.g. spatial, temporal, ...

An example (with CNN) I

From pixels to object identity:

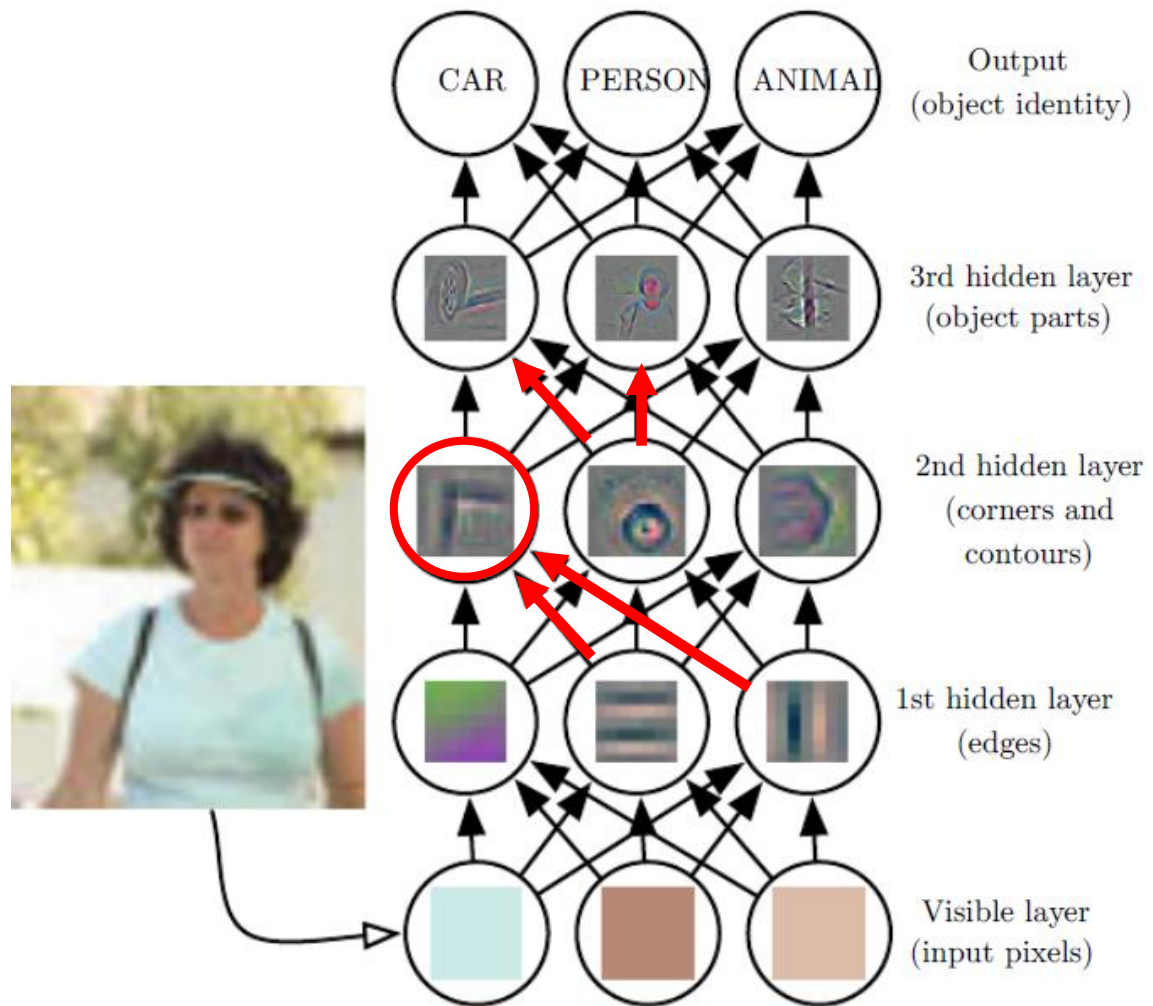
Deep learning resolves this difficulty by breaking the desired complicated mapping into a series of **nested simple** mappings, each described by a different layer of the model.

A series of *Hidden layers* extracts **increasingly abstract features from the image**.

Note: The new representation simplify the classification task at the last layer.

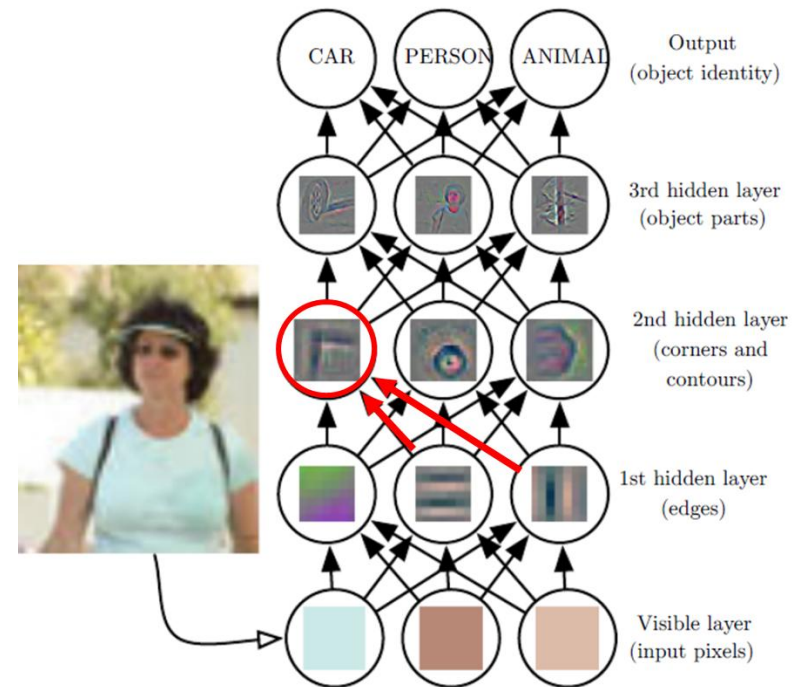
Biological visual system analogies

The images here are visualizations of the kind of feature (pattern) represented by each hidden unit

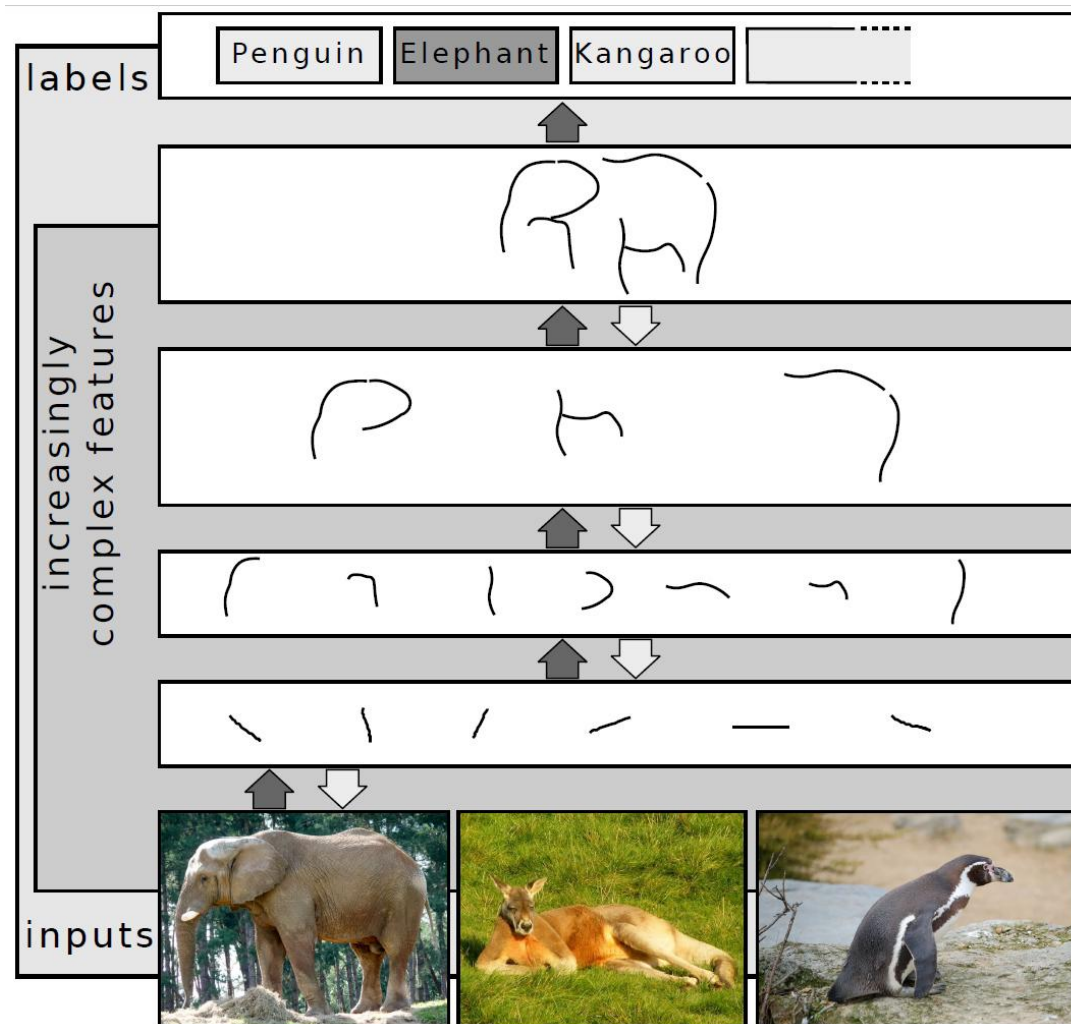


An example (with CNN) II

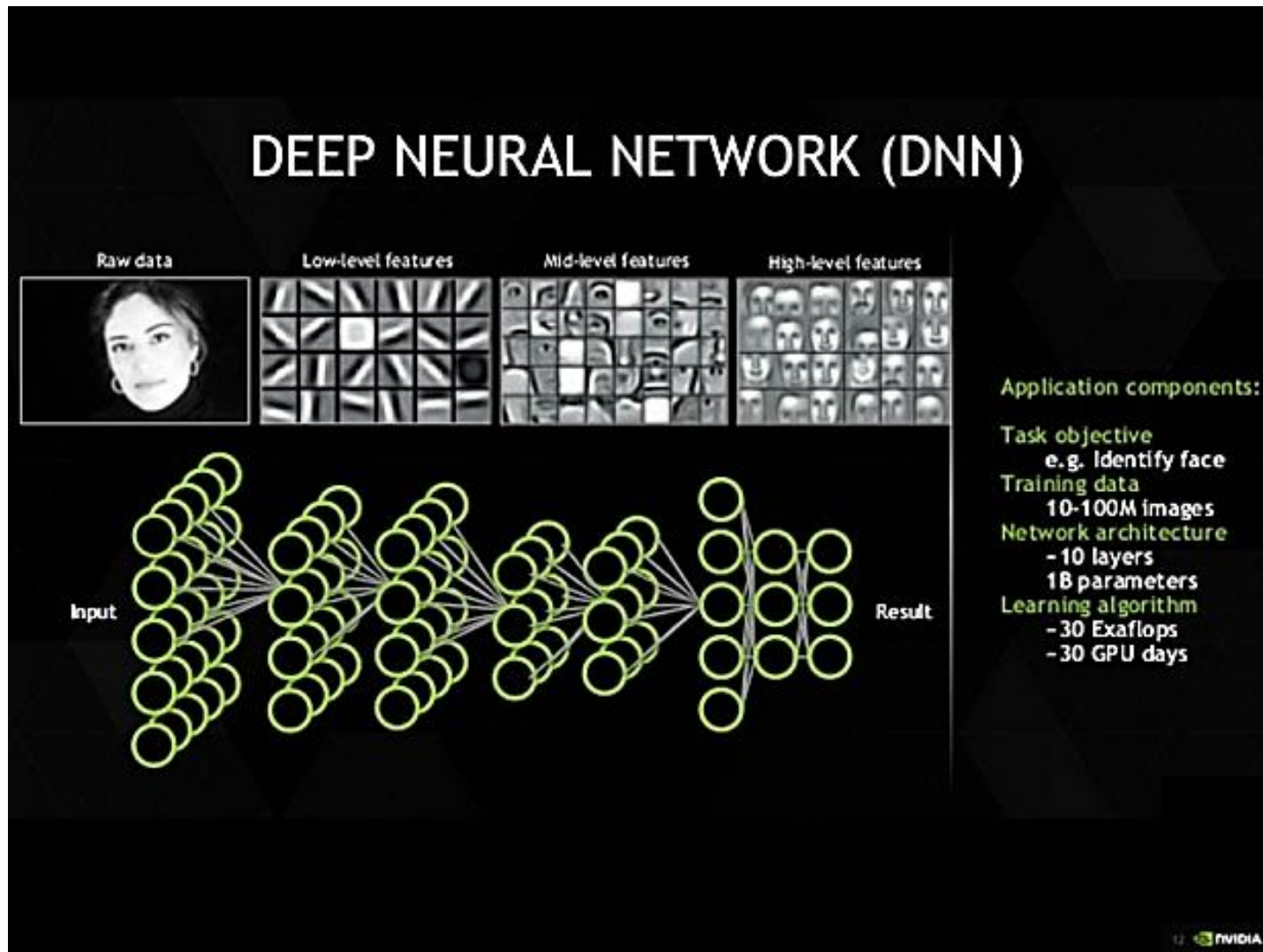
- In NN these *abstract features* are **learned** from examples!
 - **Back propagation** → the model learns how to represent what is needed to solve/help at the last layer developing a *suitable internal representation*:
Error at the output layer to better discriminate cars and persons → *delta* through the layers, the **units will specialize** accordingly to provide high level features for this, e.g. the wheel for the car, a hand for the persons, and so on back ...
 - The relationship between the automatically developed features and the reality (the inductive bias w.r.t. the nature of the language or images etc) is a **fascinating aspect** (we will discuss it later)
 - However, the **first filters** extract linear or horizontal lines as a simple *convolutive linear filter* can do. Usurprisingly we can do these as a first step.



Another example



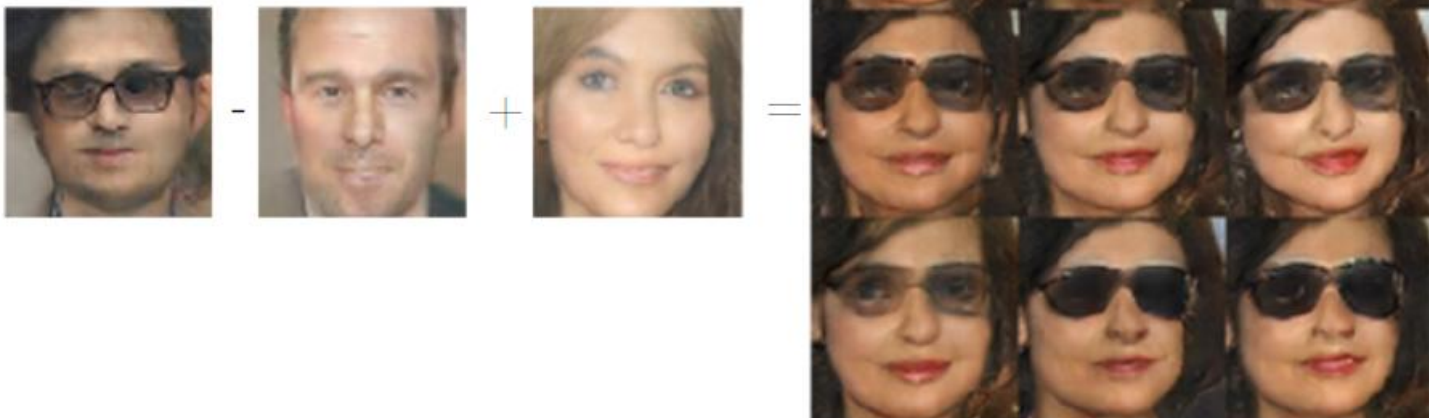
Another example



Combine to generalize (Img)

- And the abstract features can be “reused” at the higher levels
 - E.g. gender/ glasses/
 - And combine them to generalize to unseen (during training) cases:
 - Woman without glasses
 - Man with glass
- Woman with glasses can be easily represented as well.
- See the “distributed representation” section in the following

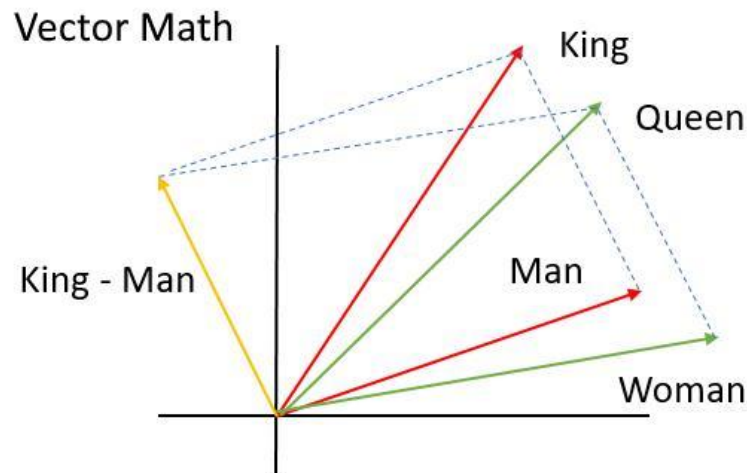
Operations on **vector representations** learned by model and then used to generate new images.



Combine to generalize (NLP)

Representation (Rep) in the vector space developed by the Deep model (inner layers)

- $\text{Rep}(\text{king}) - \text{Rep}(\text{male}) + \text{Rep}(\text{female}) = \text{Rep}(\text{queen})$ [gender relationship]
 - In this case, the vector difference between *king* and *male* captures the concept of monarchy
- $\text{Rep}(\text{Paris}) - \text{Rep}(\text{France}) + \text{Rep}(\text{Poland}) = \text{Rep}(\text{Warsaw})$ [concept of capital]
 - In this case, the vector difference between *Paris* and *France* captures the concept of capital city



Techniques: **why?** [preview]

In general, for deep learning:

- Deeper networks often are able to use less units in each layer → less parameters → Less training data to achieve a good generalization
- But many layers may be harder to be optimized (trained), hence:
- Emphasis on methods to improve
 - *Gradient descent* (also gradient vanish through layers, ...)
 - *Regularization* (due to large networks) (e.g. CNN strong architectural by w. sharing)
 - *Better exploitations of data* (e.g. semi-supervised learning, reinforcement learning cycles, multi-task learning, generative adversarial learning etc.)

.... These deserve to be presented (some of them) in the following

Also, but already discussed before:

- Larger datasets from industry and real applications (e.g. millions of images)
- Software infrastructures (also public available)
- HPC improvements (GPU and over)



Deep Learning- guideline

- Basic (or rapid introduction)
- **Insights**
 - Do we need many layers?
 - Representation learning
 - Distributed representation
- Techniques (some)
- Applications → further courses in AI (PR, NLP, ...)

Deep Learning- guideline

- Basic (or rapid introduction)
- **Insights**
 - **Do we need many layers?**
 - Representation learning
 - Distributed representation
- Techniques (some)
- Applications → further courses in AI (PR, NLP, ...)

Example from logic networks (I)



Dip. Informatica
University of Pisa

General idea (**no-flattening**):

- *when a function can be compactly represented by a deep architecture,
it might need a very large architecture to be represented by an
insufficiently deep one.*
- We will start with an example from logic circuits
- And we generalize the concept using different points of view

Example from logic circuits (II)

- A two-layer circuit of logic gates can represent any Boolean function
- Any Boolean function can be written as a sum of products, i.e. a disjunctive normal form:
 - AND gates on the first layer with optional negation of inputs, and
 - OR gate on the second layer
- What is interest for us? Our point becomes:
 - With depth-two logical circuits, most Boolean functions require an *exponential* (with respect to input size) number of logic gates to be represented [*].
 - Let us see an example

[*] I. Wegener, *The Complexity of Boolean Functions*, 1987.

The problem example: the parity function



Dip. Informatica
University of Pisa

- An extension of the XOR problem to N input variables
- Aka **odd parity: return 1 iff** there are an odd number of 1 over N binary (0/1) inputs (bits b_i or variables x_i)
- Implement odd parity Boolean function by logic gates

- Remember the XOR
$$x_1 \oplus x_2 = \underbrace{x_1 \cdot x_2}_{\text{AND}} + \underbrace{\overline{x_1} \cdot \overline{x_2}}_{\text{OR}} \xrightarrow{\text{NOT}}$$

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

- We assume to have also the NOT(x_i)
- With 2 input variables, 3 gates are sufficient

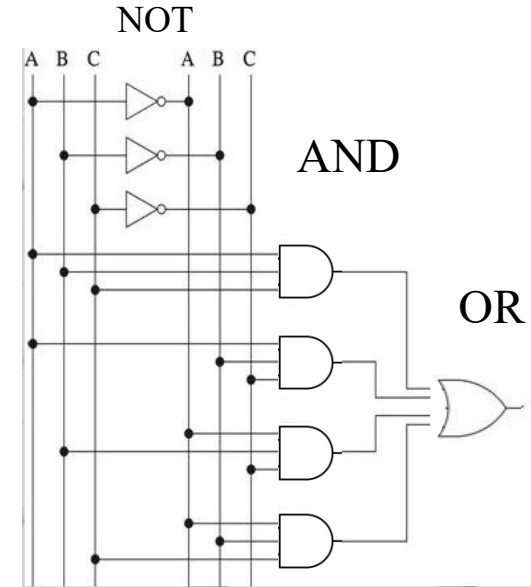
Solution with 2 layers

- For 3 inputs: 4 AND + 1 OR: **5 gates**

$$x_1x_2x_3 + x_1\overline{x_2}\overline{x_3} + \overline{x_1}x_2\overline{x_3} + \overline{x_1}\overline{x_2}x_3$$

1 iff inputs are 111 or 100 or 010 or 001

⇒ *All the combinations of the positive patterns*

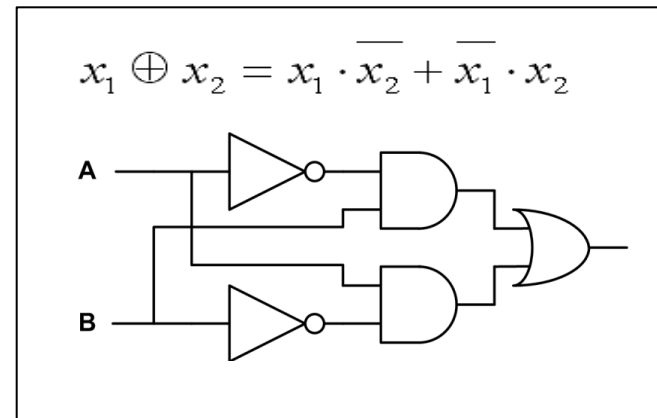
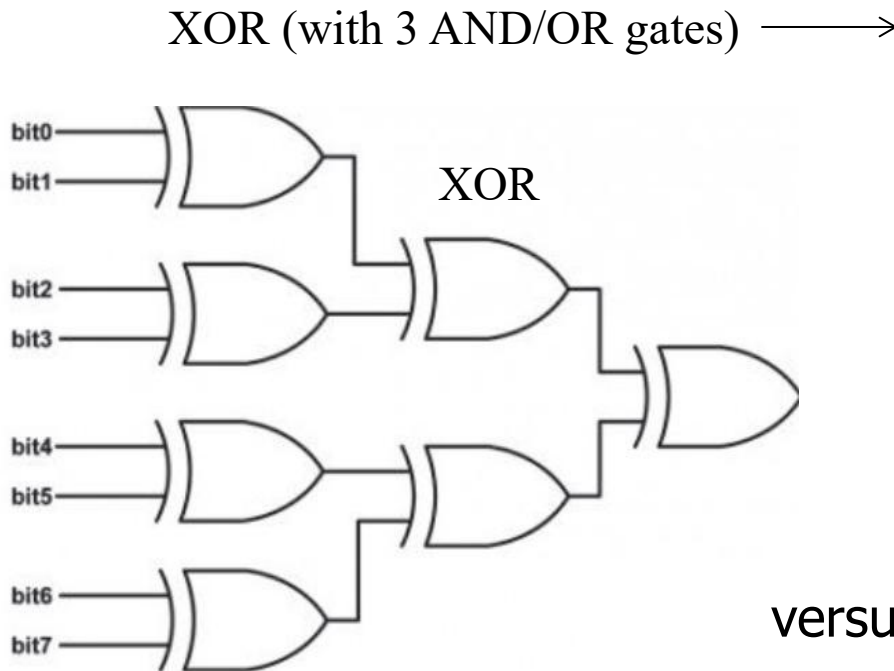


- For 4 inputs: 8 AND + 1 OR: **9 gates**
- For 8 inputs: 128 AND + 1 OR: **129 gates**
- For N inputs: $2^N/2 + 1 = 2^{N-1} + 1$ **gates**
An exponential number of gates !!!

Solution with logN layers

- Polynomial number of gates:
- complete tree* with N-1 (XOR) internal nodes
- 3 gates for each node: total 3(N-1) gates

* A complete binary tree with N leaves has N-1 internal nodes



With more layers,
for 8 inputs only
 $3 \times 7 = \mathbf{21 \text{ gates}}$
versus **129 gates** with 2 layers.

Example from logic circuits: Discussion



Dip. Informatica
University of Pisa

- However, this does not hold for all the classes of functions
- To define the class of the functions that benefits (with an exponential gain) from a multilayer structure is still an open research issue
- No-flattening theory (see later)

NN case [Repetita]: How many layers? Overview



Dip. Informatica
University of Pisa

- The *univ. approx. theorem* is a fundamental contribution
 - It shows that *1 hidden layer is sufficient* in general, but it does not assure that a “small number” of units could be sufficient (it does not provide a limit on such number)
 - Even if it is possible to find boundaries on such number (for many f. families)
- But is also possible to find “*no flattening*” results (**on efficiency, not expressivity**):
 - 1. cases for which the implementation by a single hidden layer would require an exponential number of units (w.r.t *n input dim.*), or exp. non-zero weights,
 - 2. while more layers can help (it can help for the number of units/weights and/or for learning (*) such approximation)
 - (But is it easy to optimize (training) a MLP with many layers?)

(*) Learnability: How rapidly can the neural network learn good parameters for approximating a function?

1. The case of shallow networks



Dip. Informatica
University of Pisa

REF: DL book Sec 6.4

- Cases with exp. number of units for 1-layer networks
- This is easiest to see in the binary case:
the number of possible binary functions on vectors $\mathbf{v}$ in $\{0,1\}^n$ is 2^{2^n}
- This may require an exponential number of units
 - E.g. in the worst case, an exponential number of hidden units (possibly with one hidden unit corresponding to each input configuration that needs to be distinguished) may be required.
 - See the “parity” example
- Practical issue for the dimension of the network
- But also, it implies to become difficult **to learn** with few examples for complex tasks (this is more relevant for ML)

2. No flattening of deep networks



- There exist families of functions which can be approximated **efficiently** by an architecture with depth greater than some value d , but which require a much larger model if depth is restricted to be less than or equal to d (with up to an exponential gain)
 - Examples for logic gates (1986) → see also again the example at the beginning
 - Examples for NN with standard activation functions (1990-1994)
 - Examples with ReLU (2014)
 - ...
 - ...

Theoretical analysis

Can it be proved? (open theoretical research)

- No-flattening **theorems**:

compositional functions than can be well implemented by a *deep NN* and that **cannot** be implemented retaining the same efficiency while flattening the NN (e.g. from linear to exponential number of units).

- But there is no guarantee that in general your task share such property

- A complete list of *no-flattening theorems* would show exactly when deep networks are more efficient than shallow networks

Why?

Why/when does it happen? Open line of research, e.g. essentially.....

- Deep models can exploit ***compositionality*** of internal representations:

An exponential gain in representational power, due to the fact that simpler concepts represented in a layer of the network can be exploited **as primitives** by the next layer to represent more complex concepts,

avoiding explicit *combinatorial* representation (and learning) of features

Compositionality, examples ...



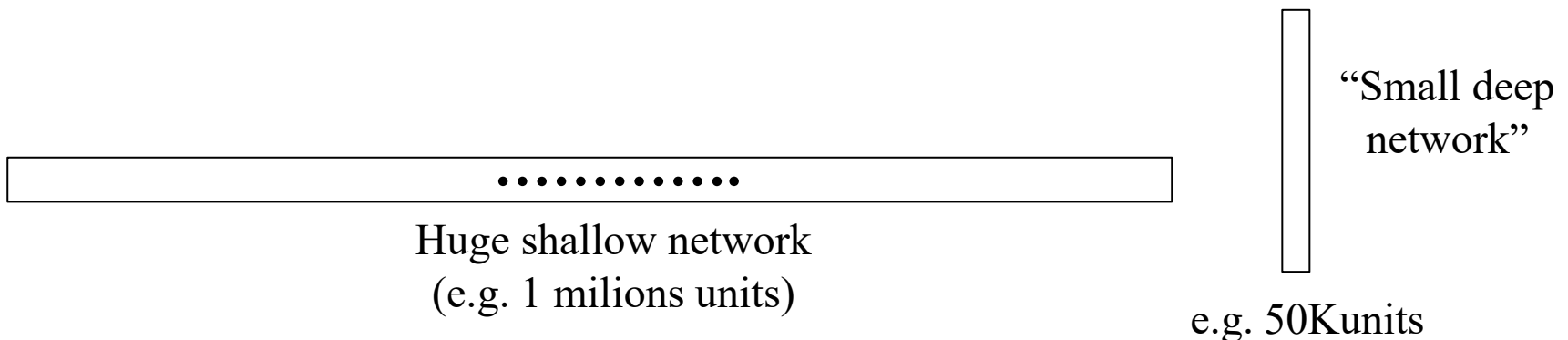
Dip. Informatica
University of Pisa

Open issue: Studies on the *compositionality of the target function* (may explain the success?)

- E.g. you have to learn each word/sub-pattern but **not directly all the possible combinations** (i.e. you can learn without having to see all the configurations), also considering them in different contexts (we will see specific examples)

Less is better!

- So, do **not** look at Deep Networks as huge model w.r.t. to small shallow models (just because you see huge instances of models for applications, *deep* is not necessarily equivalent to *big*)
- You can see them as compact models w.r.t. to potentially larger shallow models for the same task (even exponentially more compact)
- Hence, they are something less → more efficient →
- But it is **not** just for computational advantage:
- **Learning** view again: Less units and less weights **help learning** on complex task to achieve good generalization with/requiring less examples (i.e. making it feasible).



Inductive bias arguments (I)

REF: DL book Sec 5.11 and 6.4

Common aspects/**inductive bias**:

- Choosing a deep model encodes a very general belief that the function we want to learn should involve **composition of several simpler functions**.
Or nested factor of variation.
- *If our task matches this **bias** of course the deep shape of the learner is suitable.*
 - And it happens that generalization is better due to the use of many layers (see next slides examples)

Typical examples of these kinds of tasks (**hierarchical**) are for:

- The structure of images (composition of sub-graphical parts)
- The structure of language (both text and speech), also music , ...?
- New fields are under discovering

Inductive bias arguments (II)

REF: DL book Sec 5.11 and 6.4

Common aspects/**inductive bias**:

- *If our task matches this **bias** (composition of several simpler functions) of course the deep shape of the learner is suitable.*
- And even: May be that **nature** (not only images or languages) have a *hierarchical* structure itself ?
 - Multi-scale modeling, i.e. phenomena seen at different scale levels in space and time, symmetrical for rotations and translation, etc.
 - E.g. “a dog looks the same translating it” → this simplifies the task of pattern recognition and the *set of functions to be learned*.
 - “For reasons that are still not fully understood, our universe can be accurately described by polynomial Hamiltonians of low order”
Lin, Tegmark, Rolnick, D. *Why does deep and cheap learning work so well?*
J. of Statistical Physics, see the Bib.

Inductive bias arguments

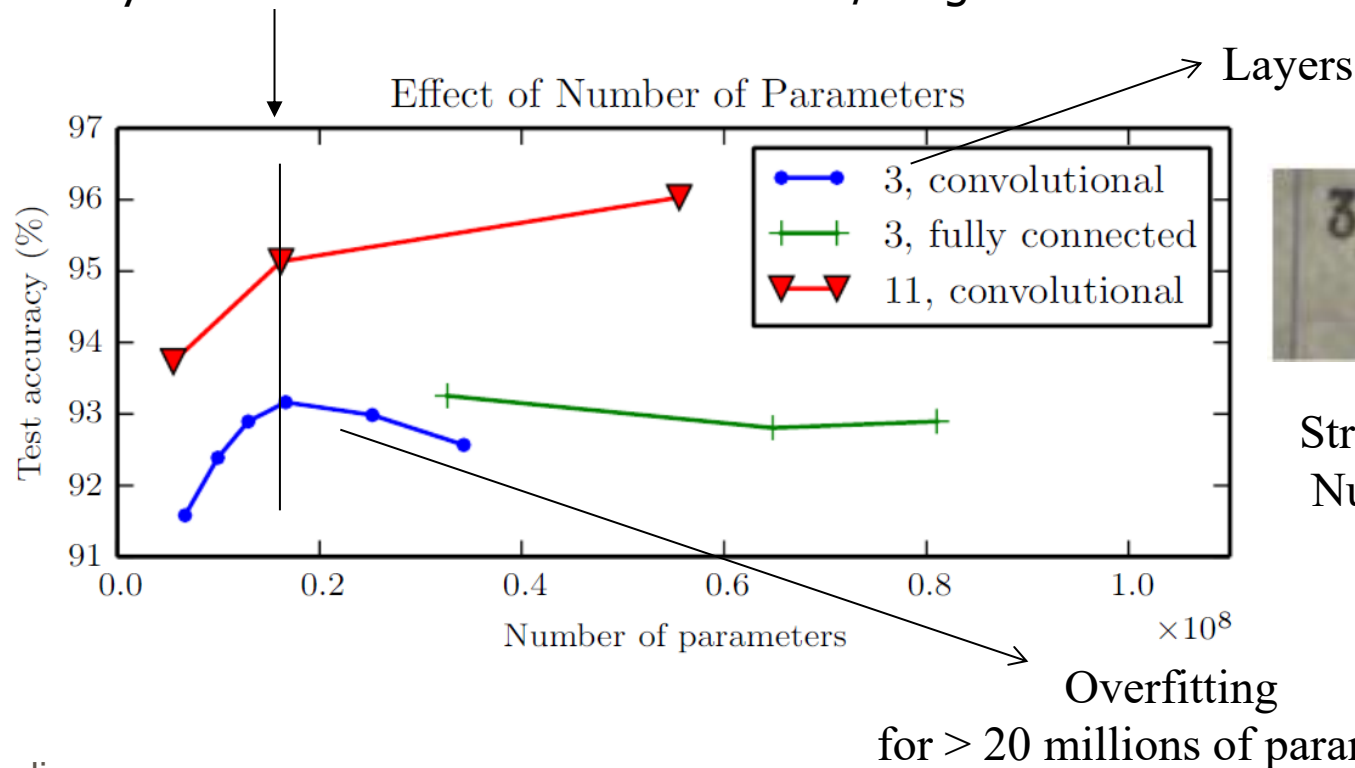
REF: DL book Sec 5.11 and 6.4

Common aspects/**inductive bias**:

- *If our task matches this **bias** (composition of several simpler functions) of course the deep shape of the learner is suitable.*
 - But, of course, **not** necessarily this holds for *all* the data and tasks
 - However, note that it is still a bias quite general,
 - much more general than other ad hoc constraints for specific tasks (e.g. specific ϕ in LBE or engineered features)
 - But, again, note that nothing is *magic* without assumptions !!!

Example: Empirical results

- *If our task matches this bias of course the deep shape of the learner is suitable.*
- In such cases deeper models tend to perform better
- And it happens, for instance, that generalization is better than a model with less layers but the same number of units/weights



Street View House
Numbers dataset,
ICLR 2014

And more arguments

There are other arguments investigating the properties of the Deep models, e.g.:

- Curse of dimensionality (number of examples needed in high dimensional space, next slide)
- Manifold learning (probability distribution for *real* images and texts seems concentrated in a manifold using only a fraction of the volume all the possible *random* images and texts, and we should be able to exploit it)

Let us see the first (even if it will clearer later discussing distributed representation advantages)

Curse of Dimensionality and Bias (I)



Dip. Informatica
University of Pisa

- Many learners rely on **local approximation** (local smoothness prior) e.g. the case of **K-nn**, **local kernels**, but also Decision Trees etc.
- But often the **smoothness assumption** (or local constancy prior) is not enough
- They need training examples to generalize surrounding and they need *many examples* to cover the entire space (that is huge in high dim., due to the *curse of dim.*), i.e. all the possible regions (when we have different target function behavior)
 - Specifically, to distinguish $O(k)$ regions in input space, all these methods require $O(k)$ examples.
- Or on the other way: Additional assumptions on the underlying data generating distribution (aka inductive bias) are needed to work with $O(k)$ examples on exponential number of regions (in k) (to generalize non-locally)
- Note that we will see again this argument in the “distributed representation” discussion

Curse of Dimensionality and Bias (II)



Dip. Informatica
University of Pisa

- One can approach the issue making strong, *task-specific assumptions* (tailored to each task, *loosing the generality*)
- **Instead**, as we discussed before, the deep learning framework chooses a *quite general* inductive bias related to composition of functions, so **again**:
 - The core idea in **deep learning** is that we assume that the data was generated by the **composition of factors** or **features** (different sub-regions representations), potentially at multiple levels in a hierarchy
- This allow:
 - to achieve a **potential exponential gain** between number of examples and number of regions that can be distinguished, as we explained before (and more later on representation learning and distributed rep.), whenever the assumption holds
 - to actually **generalize non-locally**
 - (And we find again that the **learning** benefits with less examples)

General Practical issues

- How many layers?
- How many units?

*No general
answer yet*

[→ Model selection ...]*

In general for deep learning:

- Deeper networks often are able to use less units in each layer →
→ less parameters →
→ Less training data to achieve a good generalization
- But many layers may be harder to be optimized (trained)

* A note on model selection: hyper-par of this class of models (very large NN) may be excessively high, experience makes a difference in fixing many architectural aspects.



Deep Learning- guideline

- Basic (or rapid introduction)
- **Insights**
 - Do we need many layers?
 - **Representation learning**
 - Distributed representation
- Techniques (some)
- Applications → further courses in AI (PR, NLP, ...)

Abstracting: Representation Learning

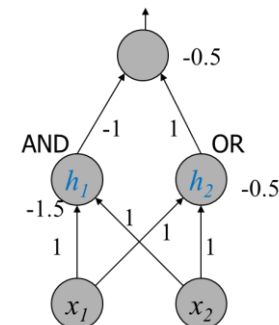
REF: DL book Chapt 15

With DL other more general concepts are emerging

- *“Representation learning is a set of methods that allows a machine to be fed with raw data and to automatically discover the representations needed for detection or classification.”*
LeCun, Y., Bengio, Y., & Hinton, G. (May 2015). *Deep learning*. Nature, 521(7553), 436-444.
- Deep-learning methods are representation-learning methods with multiple levels of representation (i.e. introduce a specific kind of reps. expressed in terms of other simpler reps).
- *But the concept is more abstract and it can be applied for many models (e.g. NN in general)*
- Note that learning the representation, or *features learning*, is in most of the cases refereed to raw data as images or text strings (not if you provide already a small set of features...)

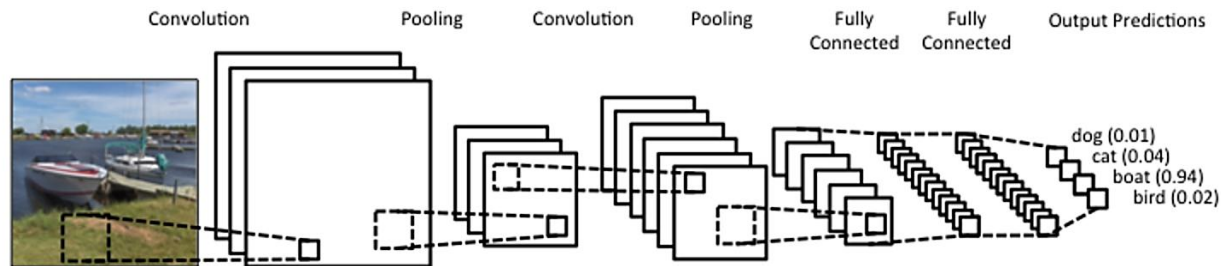
Basic ideas (I)

- Many information processing tasks can be very easy or very difficult depending on how the information is represented (everywhere in CS)
- In ML: a **good representation** is one that makes a subsequent learning task easier.
- Manual design of features is difficult, even decades of years for specific communities (language, images,)
 - A.k.a. to manually engineer the $\phi(\mathbf{x})$ of **LBE**
- Supervised learning in NN MLP is an example that leads to an automatic representation at every hidden layer taking on properties that make the output layer task easier.
 - See the XOR example in our ML course (NN1 slides)
 - A.k.a. $\phi(\mathbf{x}, \mathbf{w})$ is **learned**, finding the best $\mathbf{w}$ (see NN1 lectures)



Basic ideas (II)

- Another example:
 - In the CNN we used the *raw image* as input letting the model automatically (learn to) extract the salient features of the image for the task at hand
 - No specific pattern or engineered features have been specified in advance



- **DL** search for a representation on *multiple hierarchical levels*:
How to obtain or exploit it (beside the known full training on MLP and CNN)?

How to obtain or exploit hidden representation: Specific cases

How to obtain hidden representation (historical case):

- ❖ Semi-supervised learning: we can learn a representation for the unlabeled data and then use it to solve supervised tasks
→ ***pretraining*** approaches

How to exploit the hidden representation:

- ❖ The learned representation can be used for other tasks →
transfer learning approaches

Let me start with the first aspect: pretraining

Pretraining (preview)

“Greedy Layer-Wise Unsupervised Pretraining”

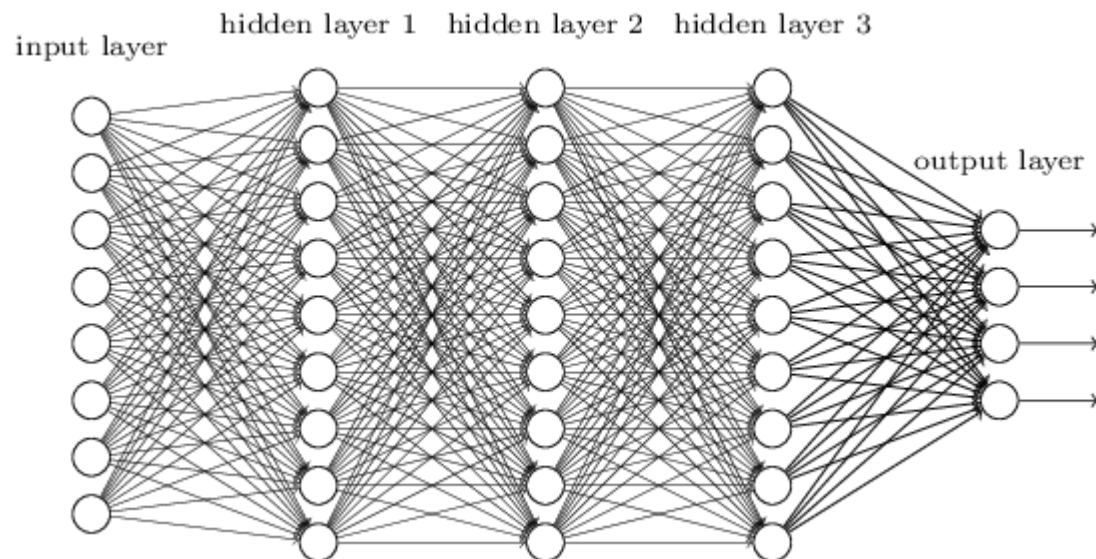
- It was the first approach to make possible to train a deep supervised networks ~2006 (other than CNN or RNN)
- Unsupervised learning as pretraining to capture the shape of the input distribution (layer by layer)
- It makes easier the training of the whole network (initialized in such way)
 - Instead of an *end-to-end* training from the output layer to the lower layers with a gradient descent (that was difficult for deep models)
- E.g. by **autoencoders** (see next slides)

Why does it help?

- ***Greedy layer-wise***: each layer is optimized independently in an ***unsupervised*** way, constituting a ***pre-training*** for the final fine-tuning of the network
- It works both in terms of:
 - Good initialization strategy
 - Regularization: herein not in terms of simpler model but in terms of discovering features that simplify the unsupervised process (regularities extracted in an unsupervised way)
 - If the true underlying functions are “complicated” and shaped by regularities of the input distribution, unsupervised learning can be a more appropriate regularizer.
 - Reducing the variance of the estimation (training) process (as pre-trained models tend to arrive in a smaller region after training)

How to build it?

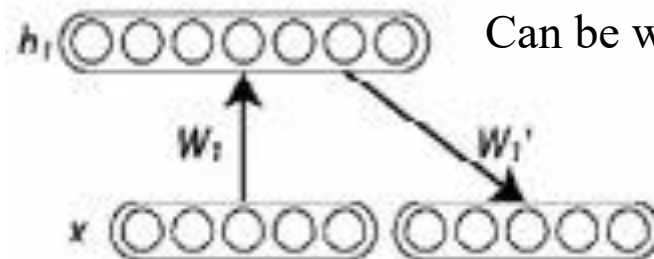
- Many approaches to implement deep learning
- Let me introduce an approach well rooted in your background through this course
- Basically by building a MLP with many layers and pre-training techniques



A premise: autoencoders

REF: DL book Sec 14.1/14.2

- An *autoencoder* is a neural network that is trained to attempt to copy its input to its output.
- Internally, it has a hidden layer h that describes a *code* used to represent the input.
- The network may be viewed as consisting of two parts:
 - an encoder function $h = f(x)$ and
 - a decoder that produces a reconstruction $r = g(h)$.



Can be with less units, see next slide

Autoencoders type

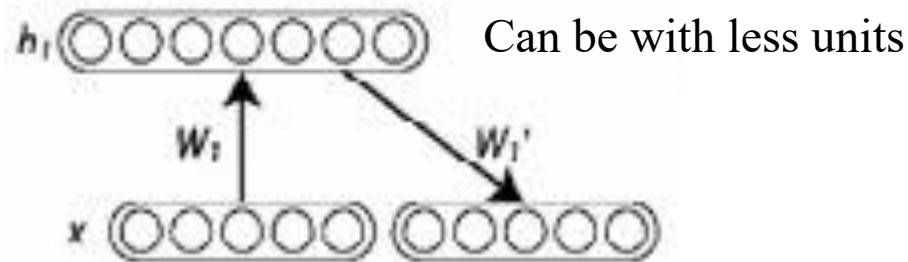
Note: there are many form of auto encoders

Also they can be

- **Undercomplete:** hidden layer is smaller than the input: force to capture the most *salient features* of the training data by architectural constraints
 - E.g. you obtain a PCA by linear decoder and a LMS approaches
- **Overcomplete:** hidden layer is grater than the input, but we use regularization to make the constraints for *sparsity* of the representation, *robustness* to noise and/or other properties of interest beside the trivial (for the over complete case) input-output “copy” capability
- Unsupervised learning!

A premise: autoencoders

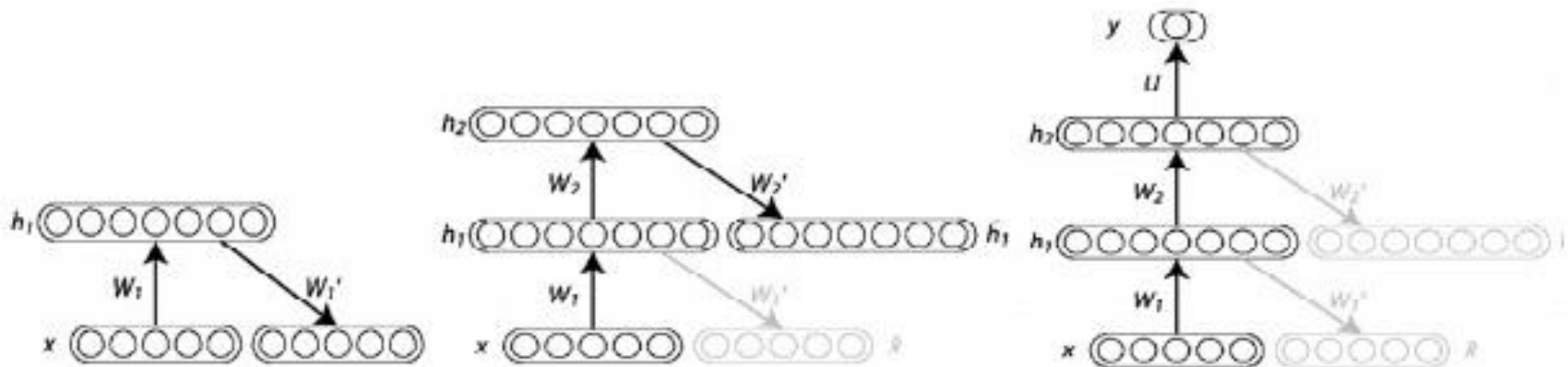
Example of use



- Usage: e.g. denoising autoencoders and auto-associative memories:
 - Training on a set of images, finding a good representation of them*
 - * “a good representation is one that can be obtained robustly from a corrupted input and that will be useful for recovering the corresponding clean input”
 - Retrieval (reconstruction) of the original image after presenting a noised version and repetitions of encoding/decoding phases

Layer-wise (pre)training

- Unsupervised (wrt to the global task): it can exploit also unlabeled data for the *layer by layer* training
- Auto-encoders used in different DL models in the form of:
 - RBM Restricted Boltzmann Machines (RBMs), which are particular energy-based models → Deep belief networks, e.g. Hinton et al 2006
 - Stacked (NN) (denoising) autoencoders, e.g Bengio et al 2007 ←

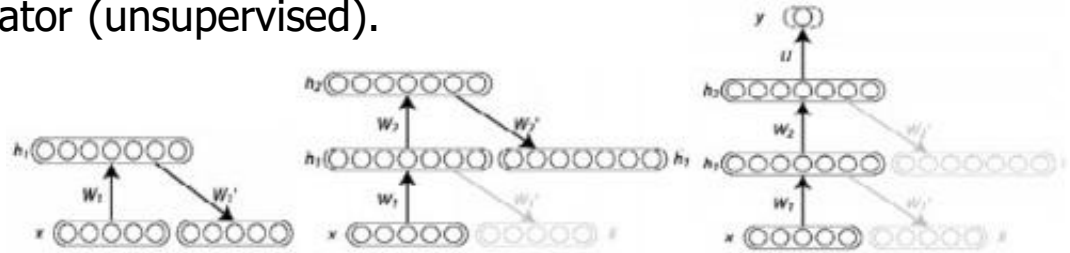


The algorithm for DL (by Y.Bengio)



Dip. Informatica
University of Pisa

1. Train the first layer as an autoassociator to minimize the reconstruction error of the raw input (unsupervised wrt the global task: we only need unlabeled examples).
2. The hidden units' outputs in the autoassociator are now used as input for another layer, also trained to be an autoassociator (unsupervised).
3. Iterate as in (2) to add the desired number of layers.



4. Take the last hidden layer output as input to a supervised layer and initialize its parameters (either randomly or by supervised training, keep the rest of the network fixed).
5. Fine-tune all the parameters of this deep architecture with respect to the supervised criterion.

The hope is that the unsupervised initialization in a greedy layer-wise fashion has put the parameters of all the layers in a region of parameter space from which a good local optimum can be reached by local descent

Note: Despite many initial successful cases, the general role of pre-training is nowadays under critical revision by researchers.

Needed?

- Pre-training can yield improvements for some tasks (especially in NLP) but not in other
- In NLP it allows to exploit large text corpora to learn (distributed) representations of words
- It allowed to start DL!
- But is difficult to be managed (e.g. effects of hyperparameters divided in two phases, pre- and full- retraining)
- “*Today unsupervised pretraining is largely abandoned*” (DL book Chap 15): the techniques for DL (see in the following) are often sufficient to make a good training from scratch
 - i.e. *end-to-end* backprop. over the deep network
- But still an open field....

Some (modern) short notes on (auto)encoders



Dip. Informatica
University of Pisa

- *Variational autoencoders* use learning algorithms in the Bayesian framework;
 - the latent space is in this case composed by a mixture of distributions instead of a fixed vector
- *Neural Machine Translation* (NMT) can be implemented by autoencoders whereas the output does not match the input - it is in another language:
 - texts are treated as sequences to be encoded into the learning procedure, while on the decoder side sequences in the target language(s) are generated.

Nowadays, we have other techniques:

- The nowadays well-know *transformers* (see [GPT](#), *Generative Pre-trained Transformer*) are encoders/decoders neural networks, plus an attentive mechanism (to learn over sequences, see a next lecture)
 - They can exploit only encoder part (e.g. BERT Bidirectional Encoder Representations from Transformers, used for various NLP tasks)
 - Or the decoder part, for generative purposes (e.g. OpenAI's GPT series for autoregressive text generation)

How to obtain or exploit hidden representation: Specific cases

How to obtain hidden representation (historical case):

- ❖ Semi-supervised learning: we can learn a representation for the unlabeled data and then use it to solve supervised tasks
→ ***pretraining*** approaches

How to exploit the hidden representation:

- ❖ The learned representation can be used for other tasks
→ ***transfer learning*** approaches

And now let see the second aspect: transfer learning

Transfer Learning

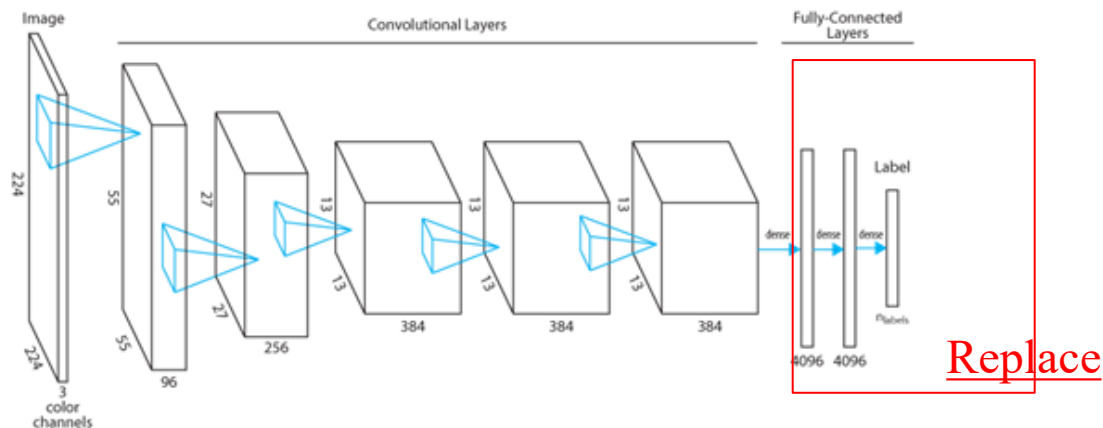
- When we use the representation discover in a model to improve another model
- We assume there exist features that are useful for the different settings or tasks, corresponding to underlying factors that appear in more than one setting.
 - Before: from the autoencoder to the classifier (a form of TL)
- ❖ But also we can use a trained model for another task (same inputs, changing the target) (**multi-task learning**) → implicit regularization
- ❖ Or changing the input domain (but sharing features), e.g. sentiment analysis for customer for videos or for customer for tv and smartphones (**domain adaptation**)
- ❖ Or take advantage of pre-trained models from a *larger datasets...*:
e.g. using AlexNet etc.



Note: these in bold are sub-fields of ML, not just terminologies!

AlexNet

- AlexNet is a Convolutional Neural Network (CNN) that has been trained on ~ 1.2 million images from the ImageNet Dataset
- With 1000 object categories, such as keyboard, mouse, pencil, and many animals. As a result, the model has learned rich feature representations for a wide range of images.
 - the primitives or basic features (low layers) are shared among different images
- The network has 5 convolutional layers and 3 fully connected layers.
- You can quickly transfer learned features to a new task using a smaller number of new training images for your new task .
- E.g. Transfer the layers to the new classification task by replacing the last 3 layers for your task and use training for these last 3 layers.
- You can fine-tune a pretrained AlexNet convolutional neural network to perform classification on a new collection of images.



Other examples of pre-trained networks



Dip. Informatica
University of Pisa

- Many pre-trained CNNs for image classification, segmentation, face recognition, and text detection are available.
- E.g. AlexNet in Matlab (Neural Network/DL toolbox)
- Also MatConvNet toolbox, VGG (Oxford) , ResNet (Microsoft), GoogLeNet, ...
- Similarly in other DL libraries (Caffe, Keras, etc.)



Deep Learning- guideline

- Basic (or rapid introduction)
- **Insights**
 - Do we need many layers?
 - Representation learning
 - **Distributed representation**
- Techniques (some)
- Applications → further courses in AI (ISPR, NLP, ...)

Abstracting: Distributed representation

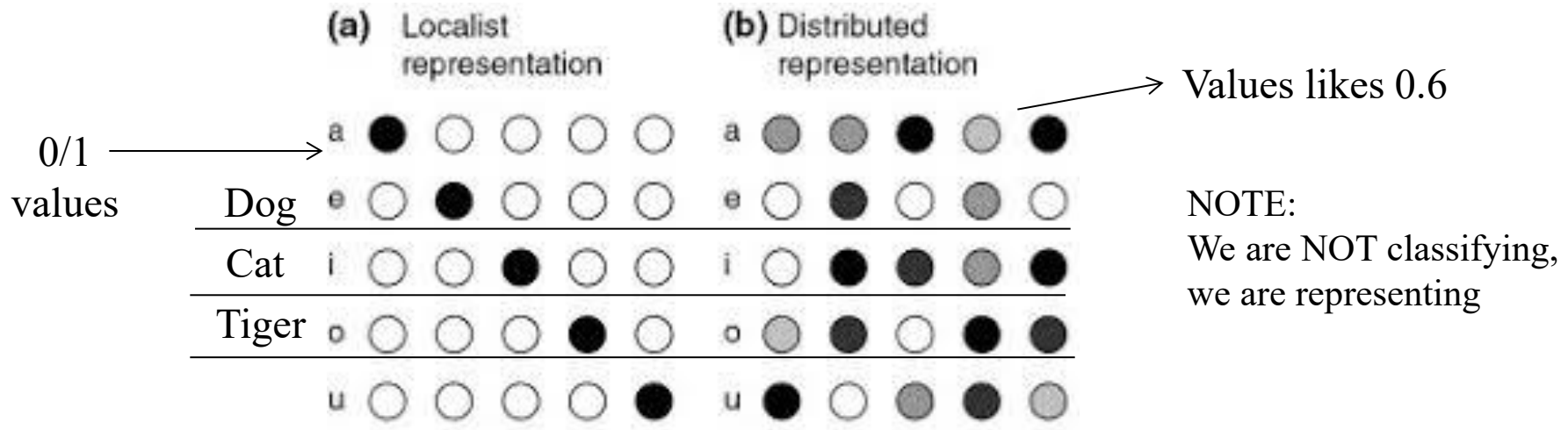


Dip. Informatica
University of Pisa

REF: DL book Sec 15.4

- *"In a distributed rep. their elements (the features) are not mutually exclusive and their many configurations correspond to the variations seen in the observed data."*
LeCun, Y., Bengio, Y., & Hinton, G. (May 2015). *Deep learning*. Nature, 521(7553), 436-444.
- Deep-learning methods exploit distributed representation with multiple levels of representation.
- *But the concept is more abstract and it can be applied for many models*, e.g. NNs in general (over hidden units) or probabilistic graphical models (multiple latent variables)*

Symbolic vs distributed



- One-hot representation
- Only one unit is 1, the other 0
- *Distance* always $\sqrt{2}$ among them

- Vectors of real numbers (dense)
- Each symbol/concept represented as the set of unit activations
- Enable to share the similarity between more than two concepts
- Can share some (*learned*) features
- The *distance* reflects the meaning (our knowledge or such learning)

- ❖ Richer and smooth representation
- ❖ Shared attributes allows generalization among different concepts(see later)

Input or internal rep.?

Question: Symbolic vs distributed refers to the input or to the hidden layers rep.?

- Here is general, but learning acts on the internal representation (representation learning aspect)
- In general, distributed rep. is used:
 - **In input** if you have background knowledge to fix it (difficult and rare)
 - Or if you fix it arbitrarily it can hamper the task solving
 - **Automatically** (*by learning*) if you can **use distribute rep. internally to the model** (e.g. hidden NN layers)
 - Typically, one-hot (not informed) rep. is used anyway for the inputs, and the model is let to develop internally the distribution rep. need for the task at hand
- Distinguish **models** able to exploit distributed rep. internally or not (see later for examples)

More in detail

We will discuss in the next few slides:

- **Count the difference** between symbolic and distributed representation
- **Shared attributes** advantages for learning and generalization, with word-embedding example

Count the difference in the discrete case (I)



Dip. Informatica
University of Pisa

- Blu car, blu bike, red car, red bike: 4 concepts
- But you do not need 4 neurons
- 2 neurons are sufficient (to represent the 2^2 concepts):
 - 1 for Blu/Red
 - 1 for Car/Bike
- and the neuron describing **redness** is able to learn about **redness** from images of cars and bikes, not only from images of one specific category of objects.

Count the difference (II)

- **Discrete case:** Distributed rep. can use n features/variables with k values to describe k^n different concepts
 - E.g. in binary code: 2^n ($K=2$) configurations versus n possible symbolic rep.s (1-hot)
- Also, e.g., if you encode by *binary* features a data of dim. N , with Distr. Rep. we can assign a unique code to exp. number of regions $O(2^n)$
- Hence ML models based on **distributed rep.** (e.g. **NN** with hidden units) is able to distinguish exponentially many more regions than the non-distributed one.
- Examples of **ML models** that rely on **non**-distributed rep.:
 - **k-nn**: 1 or few prototypes for each input: cannot share info with all the training data, and the input component cannot be controlled to be similar each other,
 - **RBF Kernels**: reduce to k-nn as seen in SVM lecture,
 - **Mixture of Gaussian**: similar,
 - **Decision Trees**: only one leaf is activated (and the path from the root)

Shared attributes: *disentangling concepts*



Dip. Informatica
University of Pisa

The same example again (*but for the meaning*)

Distribute representation

	Car (Bike)	Red (Blue)
Red Car	1	1
Red Bike	0	1
Blu Car	1	0
Blu Bike	0	0

Localist representation

Red Car	1	0	0	0
Red Bike	0	1	0	0
Blu Car	0	0	1	0
Blu Bike	0	0	0	1

Sharing a column (feature)
we *can* disentangle the concept of
“redness”
(shared among different objects)

- *Disentangling* the 2 red-blu/car-bike concepts by distributed rep. we can learn about the distinction between Car/Bike, or about the color, without having to characterize all of the configurations of the other features by examples covering all of these combinations of values.
- *Statistical separability* → make easier to *generalize* to configuration unseen during training* (see also the example on “woman with glasses” at the beginning)
 - We can easily classify blu bike having seen the other 3 cases (e.g. “I don’t like it” because not red, and the model learnt that I like red objects, with localist rep. Blu bike is completely new)

Shared attributes

We will further discuss the advantage of sharing attributes in **2 steps** (see below in **orange**)

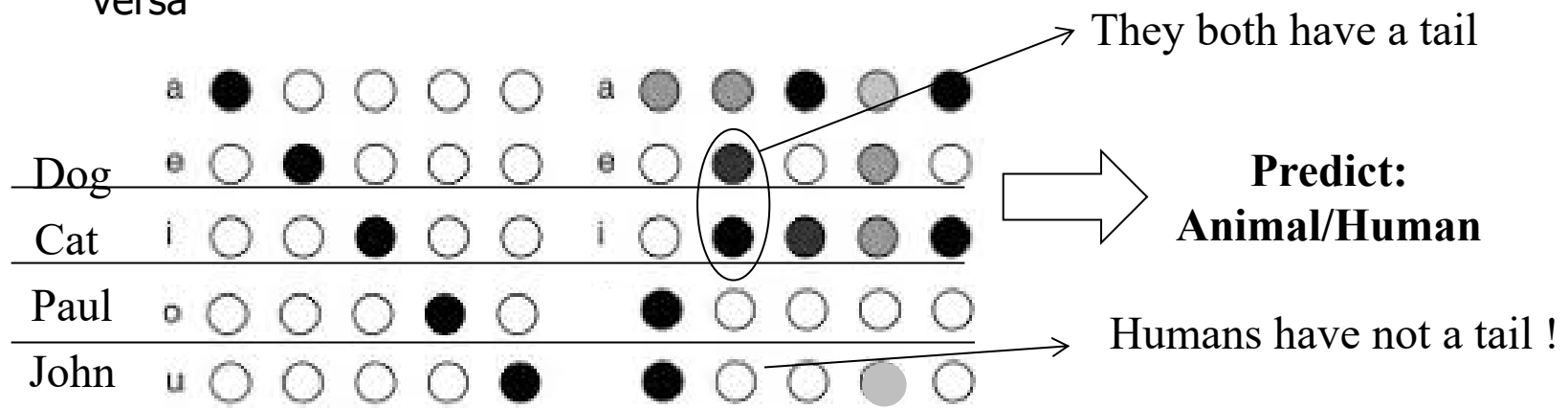
- Using examples for **word representation**:
- **Symbolic**, one-hot for each word (raw symbols have dimension equal to the vocabulary size), **VS distributed** rep. (vector space)
- Sharing some features between *dog* and *cat* words allows the model:
 1. **to treat them similarly** (when needed by the context)
 2. **to automatically learn such similarity**
- The same can be said for patterns shared in different objects, etc.

1. Treat dog and cat similarly

- Instead of using one-hot for each word (vocabulary size)
- Sharing some features between *dog* and *cat* allows the model

1. **to treats both similarly**, transferring information (by learning) from each training sentence to an **exponentially large number** of *semantically related* sentences (sharing the same features)

- For example, if *dog* and *cat* map to representations that share many attributes,
- then sentences that contain the word *cat* can inform the predictions that will be made by the model for sentences that contain the word *dog*, and vice-versa



2. Learn to be similar Word Embedding

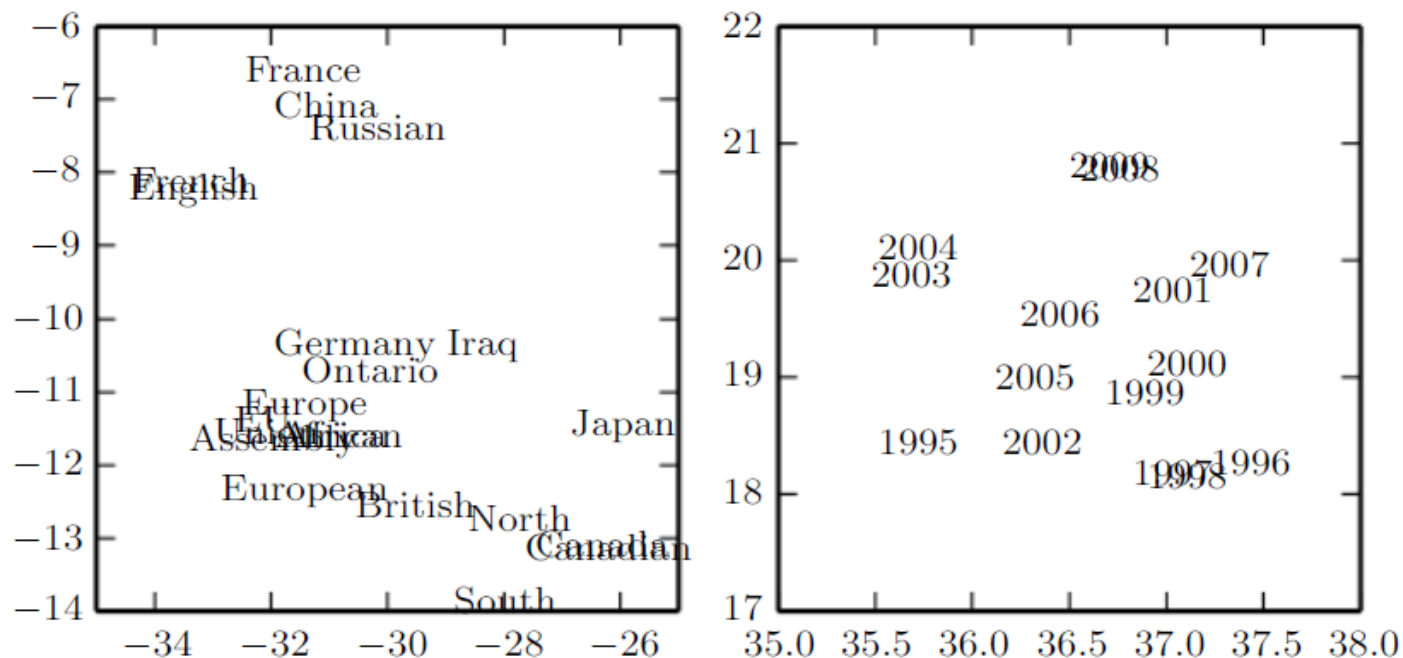


Dip. Informatica
University of Pisa

- Sharing some features between dog and cat allows the model
 - 2. to automatically make close their rep. due to similar contexts (discovering by learning *common semantic features*) :
- **Learned word-embedding :**
 - one-hot rep. of words: each word is at the same distance from each other, $root(2)$,
 - VS
 - vector space representation allowing *different similarity* between words (see for instance “word2vec” techniques)
- This often results in words with similar meanings being neighbors, preserving the semantic similarity between the words (Semantic distribution of words in the text) e.g. see next slide
- And the models can generalize much better!

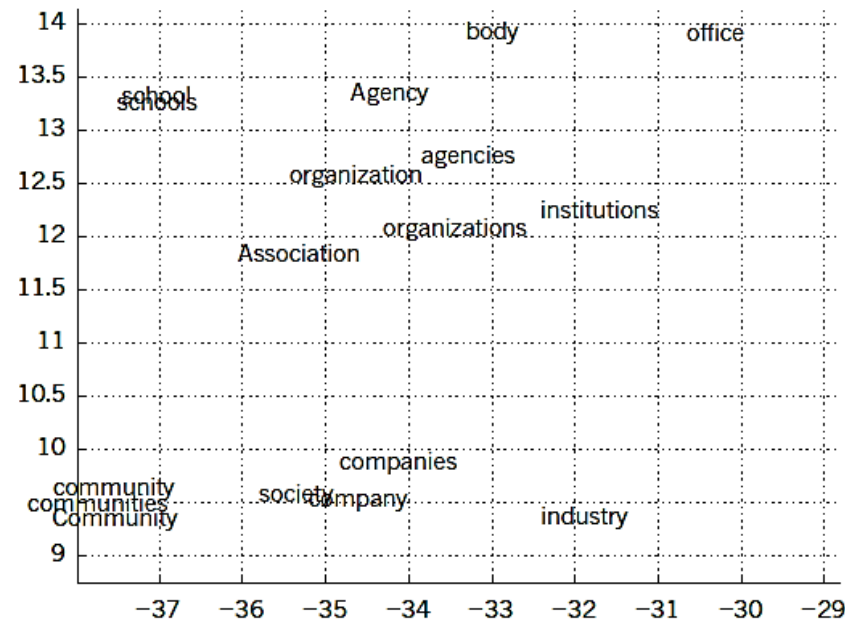
Word embedding results

Zooms in on specific areas of a ***learned word embedding space*** to show how *semantically similar* words map to representations that are close to each other, i.e. hidden layers activations projected in 2D from a *neural translation machine*: countries are in the same zone, *numbers (years)* as well

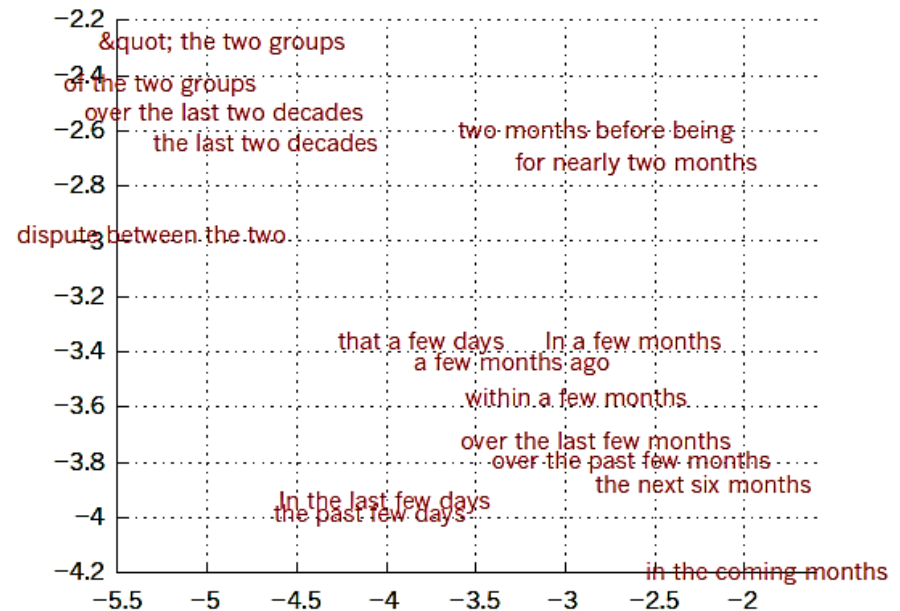


It **automatically** “discovers” *countries* or *years* from learning

Word embedding results (2)



Word embedding space (projected in 2D)



Phrases learned by an English-to-French
encoder-decoder recurrent neural network

One can observe that semantically similar words or sequences of words are mapped to nearby representations.

The distributed representations of words are obtained by using *backpropagation* to jointly learn a representation for each word and a function that predicts a target quantity such as the next word in a sequence (for language modelling) or a whole sequence of translated words (for machine translation)

Word embedding results (3)

A short discussion on DL results



Dip. Informatica
University of Pisa

- The model learns not only the grammar (this is not a surprise indeed): the learned representations are *semantically* significant
- And the model **learn them automatically**, just feeding with text used as training, **not other knowledge** is provided!
- **Issue: why our semantic corresponds to what is learnt by the (Deep) NN?**
- Probably the language (and images) intrinsically shows a (hierarchical) structure in itself and the Deep NN has the proper *inductive bias* to learn that structure (in a distributed fashion)
- E.g. real images are just a small fraction of white-noise images
- But this is still an open field of research!

Beyond NN

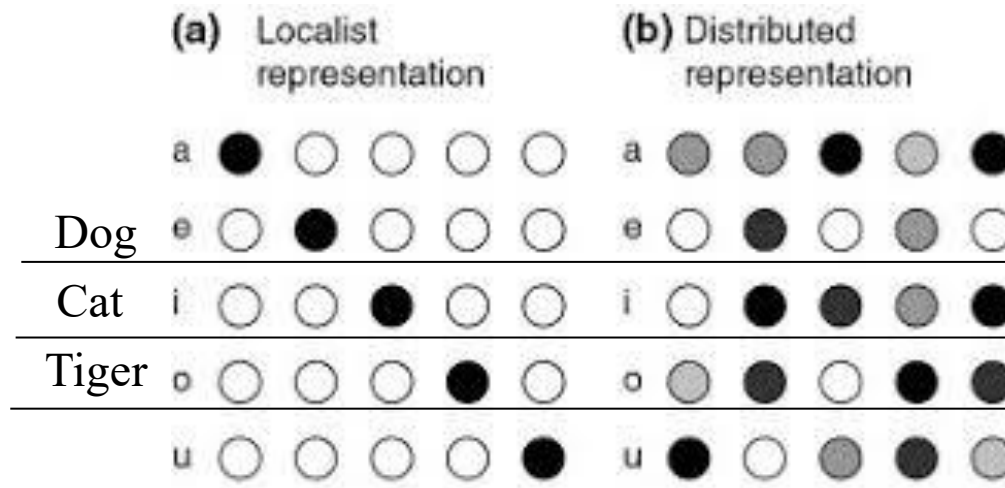
- The debate on distributed representation extend to the debate between
- **Logic-inspired** paradigms for cognition
- **Neural-network**-inspired paradigms for cognition
- For instance, recent successes in language translation raise doubts on the need of symbolic approaches to understand sentences
- ...

Note: of course, distributed rep. affects also internally the models , not just for the initial input description!

Interpretability

A drawback of the many advantages of distributed representation:

Interpretability of a *distributed representation* is **less easy**



And what for deep distributed rep.?



Dip. Informatica
University of Pisa

- DL Exploit distributed representation through many layers,
 - obtained them composing different level of abstraction
 - or by a hierarchy of reused features
- And as we already discussed such compositionality can lead to a further exponential boost to statistical efficiency (on the top to the exponential boost by distributed rep.)
 - two different potential exponential advantages (over not-distributed and shallow models).
- Globally, **DL** models learn a *distributed rep.* of data finding (or better *disentangling*) shared (by distributed rep.) causal factors that generates the data through *different level of abstraction*
- And **representation learning** is the more general concepts emerged from these studies

Combined application: Image + Text

- Combine CNN and RNN for the generation of image captions [Image source + language generation]: Impressive results, look where the attention is focused by the lighter patches generating each word (bold)



A woman is throwing a **frisbee** in a park.



A **stop** sign is on a road with a mountain in the background



A little **girl** sitting on a bed with a teddy bear.



A giraffe standing in a forest with **trees** in the background.

Vinyals, O., Toshev, A., Bengio, S. & Erhan, D. Show and tell: a neural image caption generator.
ICML [http:// arxiv.org/abs/1502.03044](http://arxiv.org/abs/1502.03044) (2014).

Medical example: skin cancer



- A deep neural network trained on 130.000 cases, much more cases than a clinician could see in (many) lifetimes.
- Capable of classifying *skin cancer* with a level of competence comparable to certified dermatologists (Nature 2017)
- It can work with an **app**

Deep Learning- guideline

- Basic (or rapid introduction)
- **Insights**
 - Do we need many layers?
 - Representation learning
 - Distributed representation
 - **Further topics** 
- Techniques (some)
- Applications → further courses in AI (ISPR, NLP, ...)

Other general insights

- **Smoothness through network structure:**
For a given number of parameters deeper networks impose more smoothness than shallow ones
 - Each layer works on the already smooth surface output by the previous layer
- Deeper networks seem to learn better, for the same number of total neurons
 - Implicit smoothness constraints
 - As opposed to explicit constraints from more conventional classification models

Recent advancements



- In the following: Recent insight topics that can deserve your attention for further (personal) studies
- Also as possible SEMINARS at the end.
- We accept volunteers ... (write to me)

Non-overfitting puzzle

- Huge NN with apparent lack of overfitting, why?
- « *A main puzzle of deep networks revolves around the absence of overfitting despite large overparametrization and despite the large capacity demonstrated by zero training error on randomly labeled data.* »
- Gradient descent enforces a form of implicit regularization
- Convergence to the maximum margin solution
- Robustness w.r.t. curse of dimensionality implications (see before)
- Poggio, T., Kawaguchi, K., Liao, Q., Miranda, B., Rosasco, L., Boix, X., Hidary, J. and Mhaskar, H., 2017. Theory of Deep Learning III: explaining **the non-overfitting puzzle**, arXivpreprintarXiv:1801.00173

Two (surprising?) theorems

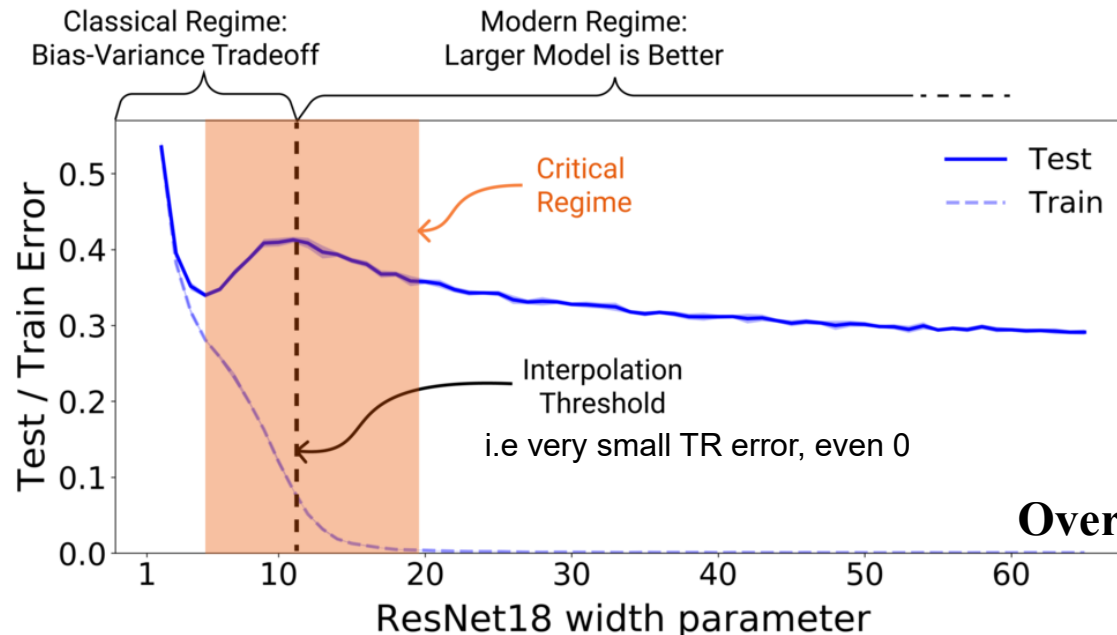
For DeepNN training:

- Variants of SGD (batch normalization and weight normalization, see later) perform minimization *equivalent to maximize (normalized) margin*
- Even standard gradient descent *maximize L_2 margin* (without explicit normalization or regularization)
(or implicitly it *controls the complexity* through an “implicit” unit L_2 norm constraint)
- Poggio et al. Theoretical Issues in Deep Networks: Approximation, Optimization and Generalization, PNAS 2019

New Insights on training large deep models



Dip. Informatica
University of Pisa



Training and test error as a function of model size, for ResNet18s of varying width on CIFAR-10 with 15% label noise.

- Deep Double descent phenomenon occurs in CNNs, ResNets, and transformers (*millions of parameters*): performance first improves, then gets worse (classical U-shape), and then **improves again** *with increasing model size*, or training time. This effect is often avoided through careful regularization.
- While this behavior appears to be fairly universal, we don't yet fully understand why it happens. *Further study of this phenomenon are an important research directions: "Effective Model Complexity" studies w.r.t number of data etc. or re-interpreting classical SLT etc.*

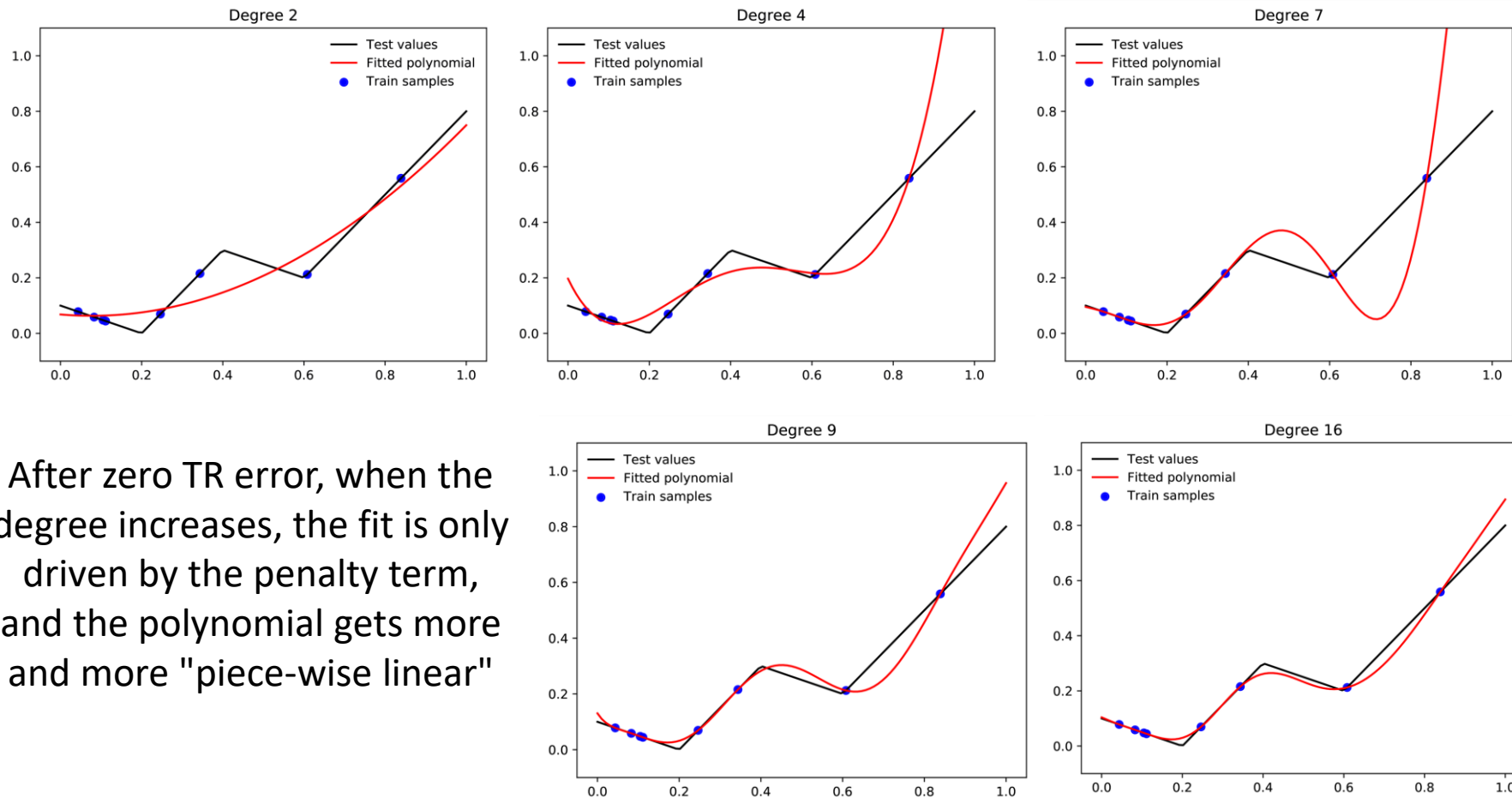
P.S. NOT necessarily in your case (small models for the prj)

A toyish example on polynomial regression with regularization (I)



Dip. Informatica
University of Pisa

Anyway, let's us as see a simple case
and that the *regularization* is still need (and in part explain the phenomena)



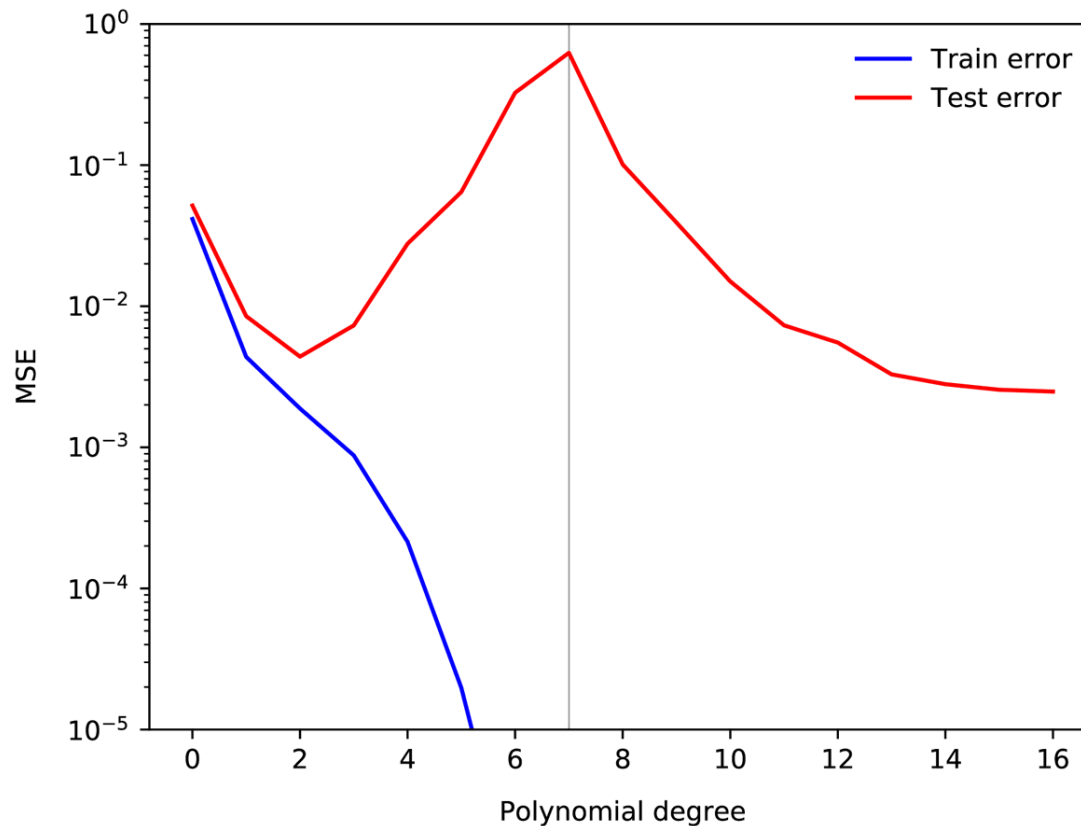
After zero TR error, when the degree increases, the fit is only driven by the penalty term, and the polynomial gets more and more "piece-wise linear"

A toyish example on polynomial regression with regularization (II)



Dip. Informatica
University of Pisa

If we look at the [median over 250 runs] of the MSEs, we get the pretty "double descent"!



Credits: François Fleuret

Much more on Double descent



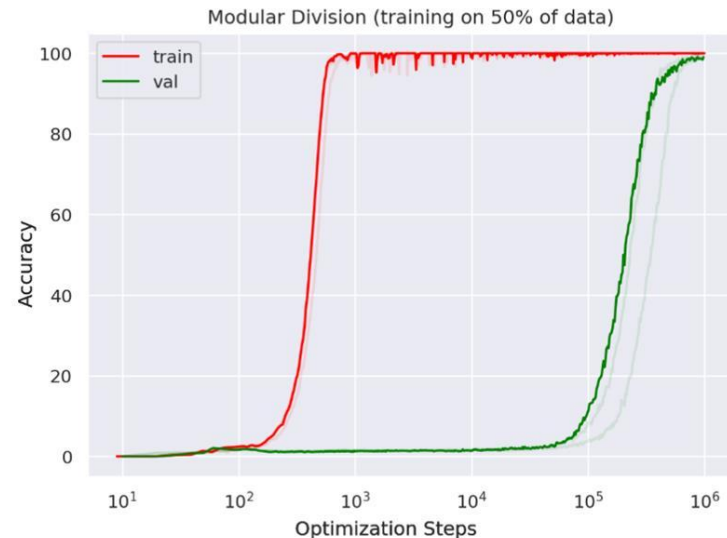
Dip. Informatica
University of Pisa

- E. H. Lee and V. Cherkassky, "*Understanding Double Descent Using VC-Theoretical Framework*," in IEEE Transactions on Neural Networks and Learning Systems, vol. 35, no. 12, pp. 18838-18847, Dec. 2024
 - or Eng Hock Lee, Vladimir Cherkassky. VC Theoretical Explanation of Double Descent,. <https://arxiv.org/pdf/2205.15549>
- L. Oneto, S. Ridella, and D. Anguita, "*Do we really need a new theory to understand over-parameterization?*" Neurocomputing, vol. 543, p. 126227, Jul. 2023, doi: 10.1016/j.neucom.2023.126227

Grokking

- Also related concepts: **Grokking** is a recently discovered phenomenon where generalization is achieved long after a model overfits to the training data (focus on the number of training steps/epochs)

For a fixed architecture, the neural network starts to generalize many epochs after reaching the interpolation threshold (i.e. when in theory it would have overfitted)



- See e.g.:
 - Light intro: Power, Burda, Edwards, Babushkin, Misra. *Grokking: Generalization beyond overfitting on small algorithmic datasets*, <https://arxiv.org/pdf/2201.02177>
 - Miller, O'Neill, Bui: *Grokking Beyond Neural Networks: An Empirical Exploration with Model Complexity*. Transactions on Machine Learning Research, 2024, <https://openreview.net/pdf?id=ux9BrxPCI8>
 - Z. Liu, E.J. Michaud, M. Tegmark. *Omnigrok: Grokking beyond algorithmic dat*, <https://arxiv.org/pdf/2210.01117>

Redundancy

- « *We demonstrate that there is significant redundancy in the parameterization of several deep learning models.*
- *Given only a few weight values for each feature it is possible to accurately predict the remaining values. [...]*
- *In the best case we are able to predict more than 95% of the weights of a network without any drop in accuracy.»*
- Denil, M., Shakibi, B., Dinh, L. and de Freitas, N., 2013. *Predicting parameters in deep learning*. In Advances in Neural Information Processing Systems (pp. 2148-2156).

Compressing the parameters

- Or compressing the parameters without accuracy loss
- Han, S., Mao, H. and Dally, W.J., 2015. *Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding*. arXiv preprint, arXiv:1510.00149.
- So, small network seems enough!

Lottery ticket hypothesis

Why, then, do we train large networks?

Frankle and Carbin answered: with the lottery ticket hypothesis:

- Good performance depends on the *lucky* initialization of one/more subnetworks.
- Large networks have exponentially more of these subnetworks.
- Testable hypothesis: Retraining pruned network on *same initialization* should provide similar performance.
- Frankle, J. and Carbin, M., 2018. The lottery ticket hypothesis: Training pruned neural networks. *arXiv preprint arXiv:1803.03635*.

Randomization

- The role of randomization (for randomly-initialized network and for random weights neural networks) is attracting more attention.
- Ulyanov, D., Vedaldi, A. and Lempitsky, V., 2017. *Deep image prior*. 2018 IEEE CVPR.
- Giryes, R., Sapiro, G. and Bronstein, A.M., 2016. *Deep neural networks with random gaussian weights: A universal classification strategy?*. IEEE Transactions on Signal Processing, 64(13), pp.3444-3457
- See also next lecture

Deep Learning- guideline

- Basic (or rapid introduction)
- Insights (do we need many layers?)
- **Techniques (some)**
- Applications → further courses in AI (PR, NLP, ...)

Techniques: why? [**repetita**]

In general for deep learning:

- We have seen many advantages of deep NN
 - But many layers may be harder to be optimized (trained), hence:
 - Emphasis on methods to improve
 - gradient descent (also gradient vanish through layers, grad. clipping,...)
 - regularization (due to large networks)
 - better exploitations of data (e.g. semi-supervised learning, reinforcement learning cycles, multi-task learning, generative adversarial learning etc.)
- These deserve to be presented (some of them) in the following

Also:

- Larger datasets from industry and real applications (e.g. millions of images)
- Software infrastructure (also public available)
- HPC improvements (GPU and over)

Our approach to the following overview



Dip. Informatica
University of Pisa

Methodological note:

- At this time in the course, it is *intentional* to do *not* add to many technical details and many other arguments to the course.
- You can deep the topic of *your* interest
- And you should have sufficient background to do it by yourself
- So, I'll just provide an overview of some interesting aspects
- (Essential) Bibliography references are at the end of each sub-part in the following
- See it in this way: I'm exploiting the idea of *distributed representation* avoiding to report all the possible combination ;-)

Key techniques for DL

For training of layered NN:

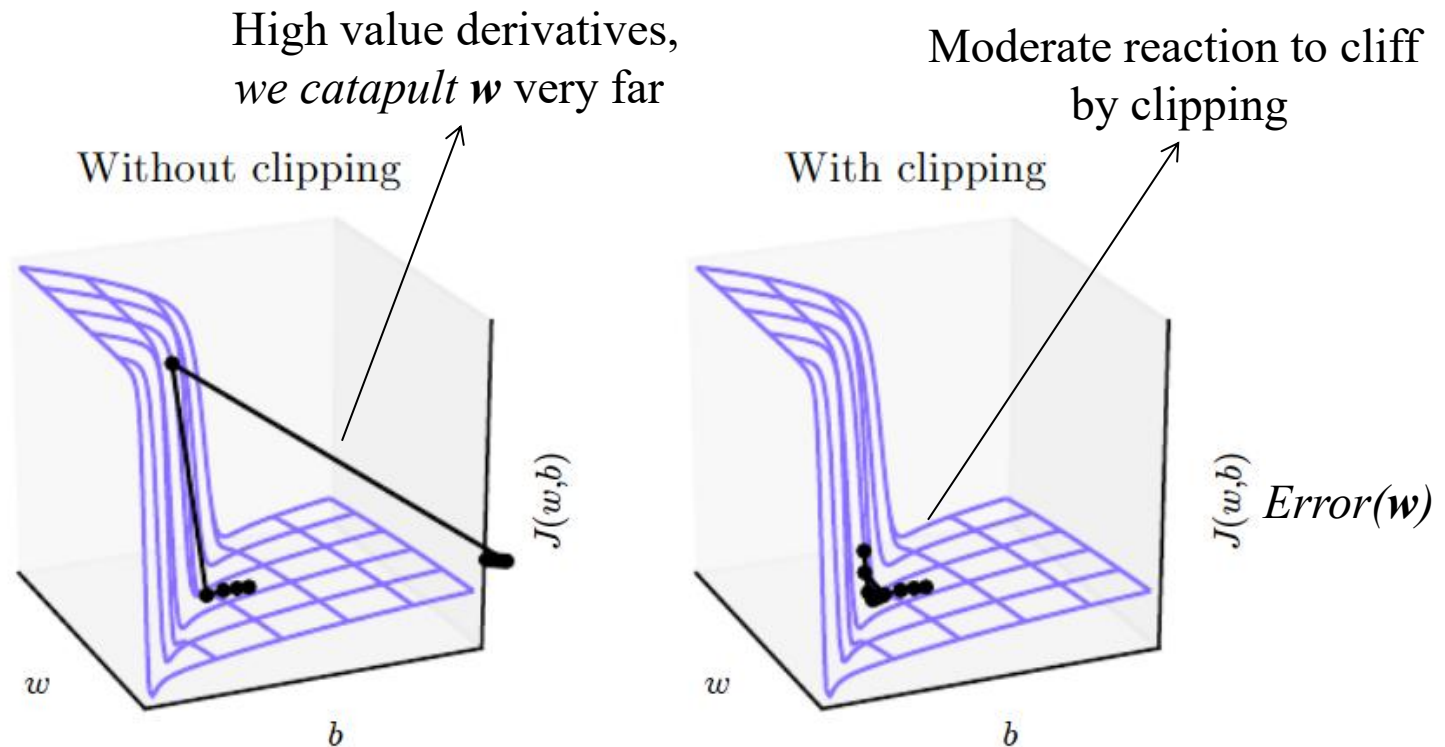
- Originally: pretraining approaches
 - But still in domains as NLP (word-embedding)
- Nowadays (in the popular libraries): *You already know them*
 - **SGD** with momentum (with decay on η , mini-batch) or Adam etc.
 - **ReLU** activation functions (hidden units)
 - Use of max log-likelihood (**cross-entropy**) loss with soft-max for output layers [also to avoid saturation/small gradients effects]
 - **Regularization:**
 - Early-stopping/Weight -decay/ *You already know them*
 - Drop-out or batch-normalization (for CNN/ sigmoid f.)

Gradient issues

- What happens to the magnitude of the gradients as we backpropagate through many layers?
 - If the weights are small, the gradients shrink exponentially (*gradient vanish issue*).
 - Especially down-sized by Jacobian of sigmoidal function
 - See also later
 - If the weights are big the gradients grow exponentially*.
- We are going to have a look to some easy approaches or discuss the issues

Clipping gradient

- Repetition of multiplication through many layers can introduce cliffs in the cost function
- Clipping: If $\|\mathbf{g}\| > \nu$ then $\mathbf{g} = \nu \mathbf{g} / \|\mathbf{g}\|$, $\mathbf{g}$: gradient and ν : norm threshold
So to move in the gradient direction, but bounding the weight update



Gradient Vanish: a note

Most of the heuristics try to deal with the problem of **gradient vanish** that hampers the training in early deep NN in the low layers:

- Traditional activation functions such as the *hyperbolic tangent* function have gradients in the range $(-1, 1)$, and backpropagation computes gradients by the *chain rule* (repeated through the layers).
- In d -layer network, this has the effect of multiplying d of these small numbers to compute gradients of the "front" layers (close to inputs),
- meaning that the gradient (error signal delta) decreases exponentially with d while the front (close to the input) layers training becomes very slowly.
 - Of course if weights grows the backpropagated signals can increase (too much? Exploding case, but much more rare for tanh/sigmoidal)

There are many **approaches** to deal with it:

- From Rprop, to short-cut connections, up to randomized NN (see next slides) (see also further course for the RNN case)
- Let see the new ones (ReLU, ...)

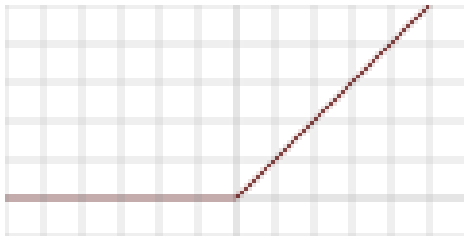
Sketch of the idea [tech#]

- Let M last layer, l current layer
 - $\mathbf{h}_l = F_l(\mathbf{h}_{l-1})$
 - $F = F_M \circ \dots \circ F_1$ the layers composition
- $\rightarrow \partial F / \partial w_l = \partial F_M / \partial \mathbf{h}_{M-1} \cdot \dots \cdot \partial F_l / \partial w_l$
- $\rightarrow |\partial F / \partial w_l| \approx \rho^{M-l+1}$ assuming $|| \partial F_i / \partial \mathbf{h}_i || \approx \rho$
- $\rho \leq 1 \Rightarrow$ vanishing/exploding of the gradient
 - We can discuss it at a Q&A

ReLU activation function

REF: DL book Sec 6.3

- ReLU (Rectified Linear Unit):

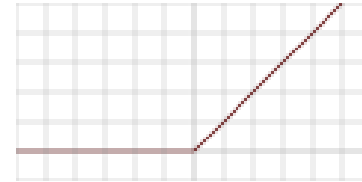


$$f(x) = \max(0, x)$$

$$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$$

- Y. LeCun, L. Bottou, G.B. Orr and K. Müller (1998). "Efficient BackProp"
<http://yann.lecun.com/exdb/publis/pdf/lecun-98b.pdf>
- X. Glorot, A. Bordes, Y. Bengio (2011). *Deep sparse rectifier neural networks*
- In DL BOOK: Section 6.3.

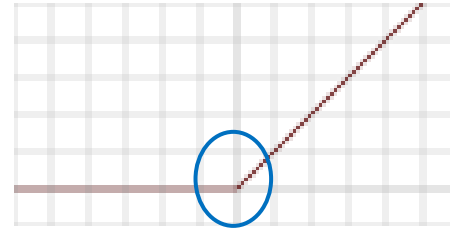
ReLU: Discussion (I): f'



- Why use it for DL (since 2009)?
 - Simplified/Better gradient propagation through many layers (see previous slide for gradient vanish)
 - Avoid **saturation** effect of **sigmoidal functions**, where **gradient** shrink to small (see NN-part1: always $f'(x) < 1$), **while ReLU always has $f'(x) = 1$ for positive net**
 - Hence the gradient can be better propagated back through many layers, memento:

$$\boxed{\delta_j} = \left(\sum_{k=1}^K \delta_k w_{kj} \right) \cdot f'_j(\text{net}_j) \quad \text{if } t = j \text{ (hidden unit)}$$

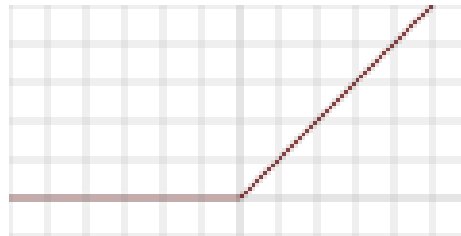
ReLU: Discussion (II): In 0?



- Gradient in 0? Warning: It is Not differentiable in 0
- But often (in ML libraries) is assumed simply to be 0 (left derivative) or 1(right deriv.): approximation is “acceptable and safe” because
 - already input/numerical approximation make evaluation on $f(0)$ unlikely to be really for a true value $x=0$.
 - So $f'(x) = 0$ if $x < 0$, $=1$ if $x > 0$, and arbitrarily 0 or 1 in zero.
- To be more principled: CM approaches for PieceWise Linear functions (via Non-differentiable optimization, etc.)

ReLU: Advantages

- Efficient gradient propagation: w.r.t. vanish (previous slide)
- Sparse activation: E.g., in a randomly initialized network, only about 50% of hidden units are activated (having a non-zero output).
- Efficient computation: only comparison, addition and multiplication.
- Still neuroscience inspiration (*Glorot et al, 2011*)
- Faster and effective training of **deep** neural architectures

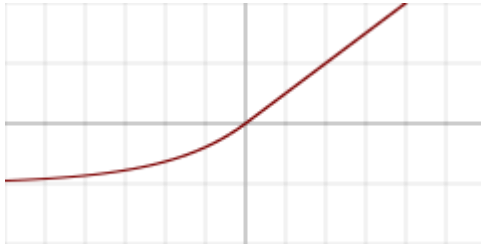


Also for ReLU

- Tricks:
 - Start with positive nets , e.g. bias to a small, positive value, such as 0.1.
 - This makes it very likely that the rectified linear units will be initially active for most inputs in the training set and allow the derivatives to pass through.
- Many generalization (see also the NN –part1 lecture and/or sec 6.3.1 DL book)
- An updated list (Note: many perform comparably!):
https://en.wikipedia.org/wiki/Activation_function
- In general they avoid some drawbacks but still retaining the linear part, which in general made the model easier to be optimized...

Examples beyond ReLU

- ELU (Exponential linear unit)



$$f(\alpha, x) = \begin{cases} \alpha (e^x - 1) & \text{for } x \leq 0 \\ x & \text{for } x > 0 \end{cases}$$

- Leaky ReLU



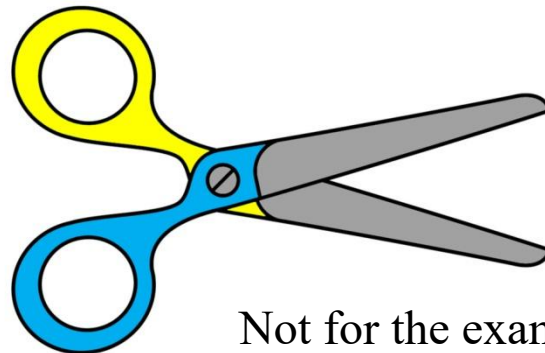
$$f(x) = \begin{cases} 0.01x & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$$

- Learning also on the left part (neurons are not turned off), mean activation closer to zero, ...
- An updated list https://en.wikipedia.org/wiki/Activation_function

Other

- Hessian Free Optimization (already discussed)
- Smart Initialization: pre-training techniques
 - Pre-training, aka "A good start is half the work"
 - Drive weights in a region where it is easy to reach better (local) minima
 - e.g. auto-encoders seen before

If we will have enough time we will discuss details later



Not for the exam
(unless you use it)

Batch normalization - Overview



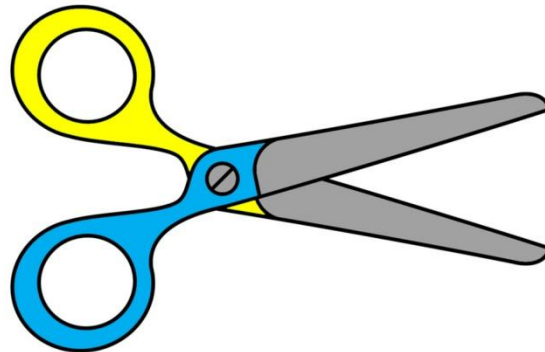
Dip. Informatica
University of Pisa

- **Batch Normalization** is a method that *normalizes* each batch by calculating each individual batch statistic such as mean and variance for each layer (reparametrization).
 - Normalize each matrix [databatch x activation of units] with mean and variance (shifting values to zero-mean and unit variance)
 - Include it in backpropagation
- As normalizing the input is a standard approach, BN helps by making the data flowing between intermediate layers of the network to stay normalized.
 - Regularization effect (due to the noise added at training time by the mini-batch size)
 - Achieving faster learning and higher overall accuracy for DL (it deserves to be mentioned due to its relevance to allow CNN/sigmoidal deep models to be efficiently trained).
- To deepen: DL book Sec. 8.7.1. or ICML 2015 (Ioffe, Szegedy, Batch Normalization: *Accelerating Deep Network Training by Reducing Internal Covariate Shift*).

Batch normalization - Details

Include it in backpropagation: <http://kratzert.github.io/2016/02/12/understanding-the-gradient-flow-through-the-batch-normalization-layer.html>

If we will have enough time we will discuss details later



Not for the exam
(unless you use it)

Dropout - Overview

REF: DL book Sec 7.12

- Dropout is a method that randomly selects a subset of the network *during its training*
- It combines (can be explained in terms of)
 - Ensembling (implicit bagging, different sub-networks)
 - But computationally inexpensive
 - Regularization
- “*Dropout: a simple way to prevent neural networks from overfitting*”
JMLR 2014
- Section 7.12 DL book

Concepts that
you already
know!

From Bagging to Dropout

BAGGING (REPETITA)

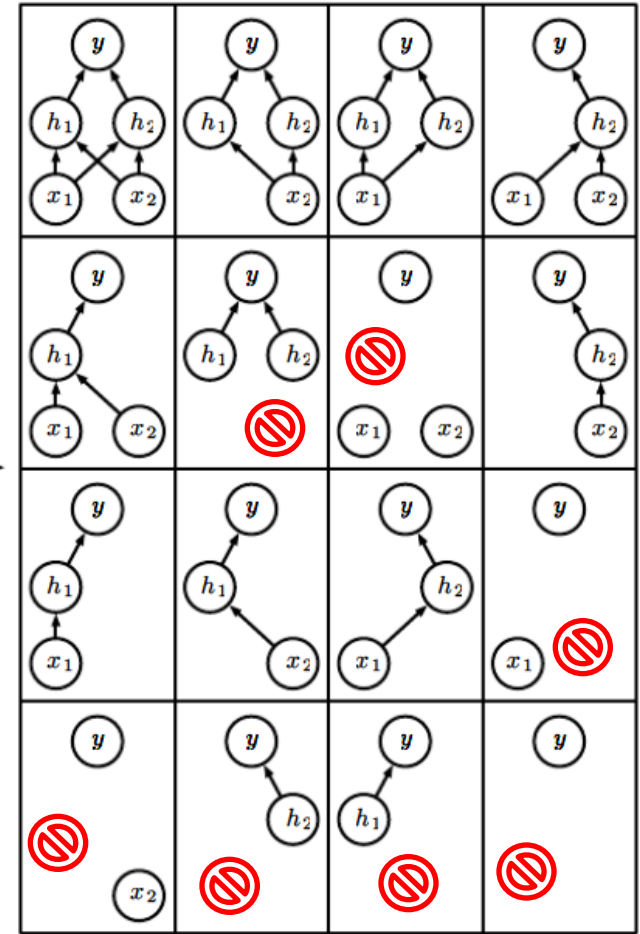
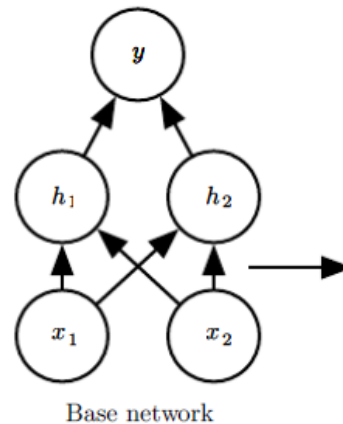
- Combine many classifiers/models
- Bagging
 - Train n classifiers/models on different subsets of training set
 - Differentiate each training using bootstrap (resampling with replacement)
 - Output the average outcome
- High variance models (with low bias) can perform well on average

DROPOUT aims to approximate (and exploit) this process,

- but with an exponentially large number of (sub) neural networks
 - while at test time we will have still only a single network!
- and also aims to maximize the diversity of the ensemble preventing complex co-adaptations on training data

Dropout: picture

- Dropout trains the ensemble consisting of all sub-networks that can be formed by removing non-output units from an underlying base network
- In the example some networks cannot work 
- With larger network the probability of no path from input to output is low.



Ensemble of Sub-Networks

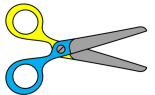
Dropout Basic Algorithm

- Each time we load an example into a minibatch, we randomly sample a different binary mask to apply to all of the input and hidden units in the network.
 - The mask for each unit is sampled independently from all of the others.
 - The probability of sampling a mask value of one (causing a unit to be included) is a hyperparameter fixed before training begins. Typically, e.g. **0.8** for input units, **0.5** for hidden units
 - Each unit in the network is multiplied by the corresponding mask, (zero is like removing, "**drop out**", the unit → build sub-networks)
- We then run forward propagation, back-propagation, and the learning update as usual (training a subset of units at time).
- The removed units are then reinserted into the network with their original weights.

Dropout insights

- **Parameter sharing** among sub-networks: only a small fraction of the possible sub-networks are each trained for a single step (many are not explicitly trained per-se), but the *parameter sharing* causes the remaining sub-networks to arrive at good settings of the parameters.
 - Aka Many paths sharing the same links....
 - This allows us to treat an exponential number of models with a tractable amount of memory
 - 10-20 masks are typically enough!
- **Approximation to the predictions of the entire ensemble:** 

To do it better, *weight scaling inference rule*: multiple the weights with probability of inclusion (typically 0.5) to estimate the expected unit output during forward phase

 - Simulate what happens during training for each unit input (expected value of the output of any unit is the same as in the training stages)
 - No other details.... 

Dropout and Regularization

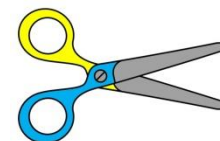
- Dropout regularization effect:
 - avoiding training all units on all training data, reduce units interactions (and diversify as useful for committees)
 - variance reduction as for bagging
 - insert structured noise (recognize a face with the hidden unit for the nose, compare with standard regularization by adding deformed inputs with random noise)

Moreover, interestingly it *regularizes* each hidden unit to be not merely a good feature but a feature that is good in many contexts (different sub-networks).

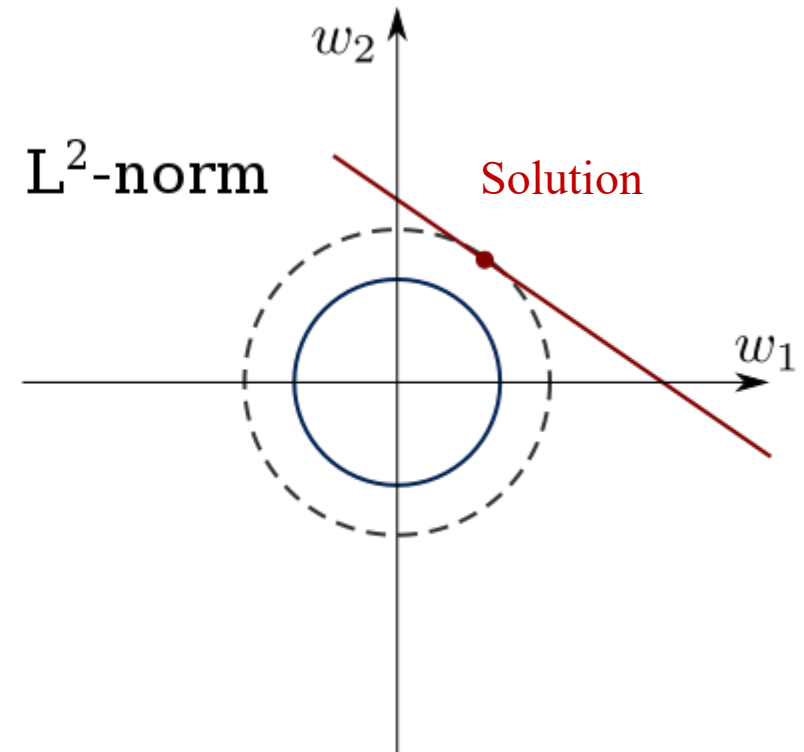
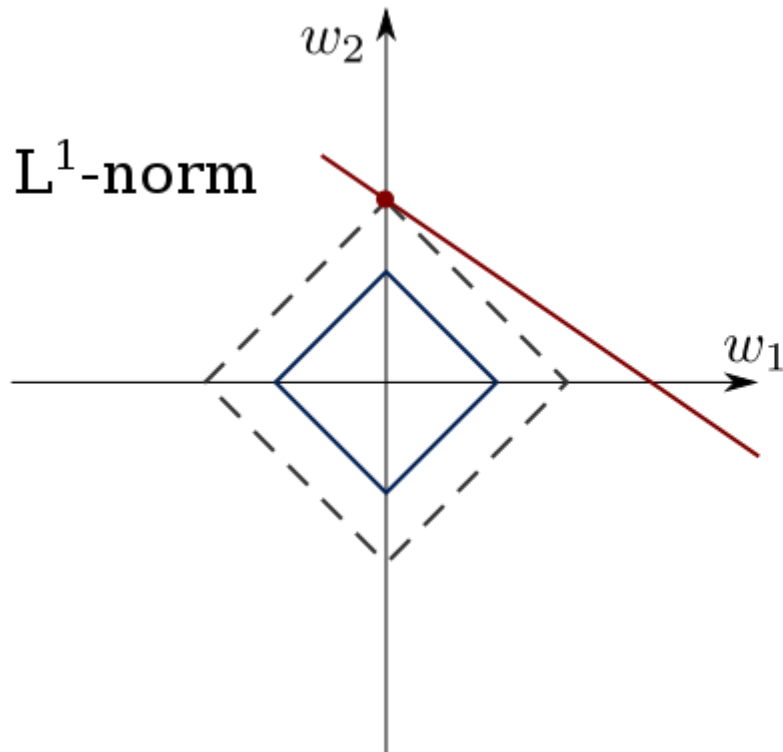
- It can be show that for linear regression is equivalent to L2 weight decay, using different lambda for each input [Wager et al. 2013]
- Can be used for any model that uses *distributed representation* and SGD training
- Can be extended to any kind of random modification (not only to exclude units).

L1 regularization

- Use L1-norm (sum of abs. values) instead of L2-norm in the penalty term
- LASSO alg. for linear models: it favorites *features elimination* (weight zero) wrt to L2 (which often only shrinks the size of the weights without setting any of them to zero) producing a simpler final models
- Solution can be approximated but in general requires non-differentiable optimization techniques.
 - the extension to non linear-modes, like NN, needs special approaches, e.g. smooth approximation of the L1 norm, etc
 - nondifferentiable optimization, sub-gradient, smoothed gradient approaches ... can be seen in the CM course
- This is a general topic NOT related to DL (we left it many lectures ago).
- For this year we refer to the CM course



L1 vs L2 regularization



Adversarial Learning

- Adversarial training
- **Generative adversarial network**
 - two neural networks contesting with each other
 - one network generates candidates and the other evaluates them.
- E.g. 2017: astronomical images and many other applications
- An example of possible SEMINAR at the end.



GAN producing photorealistic images of imaginary celebrities

These people do not exist!



- https://research.nvidia.com/publication/2017-10_Progressive-Growing-of



Deep Learning- guideline

- Basic (or rapid introduction)
- Insights (do we need many layers?)
- Techniques (some)
- **Applications** → further courses in AI (ISPR, NLP, ...)
- And **Discussion**

Successful applications in ...

Aka consider DL for

- NLP (including speech recognition, machine translation, sentiment analyses, question answering...)
- Computer vision (processing images and video, with many tasks including object recognition, and many many applications areas from medical applications to image synthesis, ...) with performances never achieved with previous methods.
- GO game solved with DL
- More at the **NLP** and **ISPR** courses (e.g. in the AI curricula)

Challenges

- Theoretical analysis and framework
- Optimal architecture design, e.g.
 - How many layers? Which type of units is the best? ... (as empirical CV approach could be excessively expensive in this context)
- Efficiency for big data
- Processing streams of data (on-line)
- Structured data (sequences, trees, graphs)
- ...

DL conclusions

- The recent **technical/data** advancements made possible to learn with deep and very large NN/models
- Many **successful applications** have been developed, whereas DL provides the state-of-the-art solution (especially where compositionality for features is relevant: visual, language, ...)
- The insights suggest (two different) **potential advantages** (even exponential) in the representation and learning capability of such construction
- The **theoretical studies** and the extension of the **conceptual frameworks** (e.g. representational learning) and of the **models** and **applicative domains** are growing at an unprecedented rate
- Globally, it is becoming more and more evident how Machine learning makes it relatively easier to develop sophisticated AI (but more in general software) systems working for **real-world solutions**

Bibliography (DL)

General ML course biblio:

- I. Goodfellow, Y. Bengio, A. Courville: ***Deep Learning***, MIT Press, 2016 . On-line: <http://www.deeplearningbook.org/> (see sections in the previous slide references, DL book)

Other readings:

- Light introduction: LeCun, Y., Bengio, Y., & Hinton, G. (May 2015). *Deep learning*. Nature, 521(7553), 436-444.
- Not formal but intriguing: Lin, H.W., Tegmark, M. & Rolnick, D. *Why does deep and cheap learning work so well?* Journal of Statistical Physics (September 2017), 168, 1223-1247.
Also: <https://arxiv.org/abs/1608.08225>

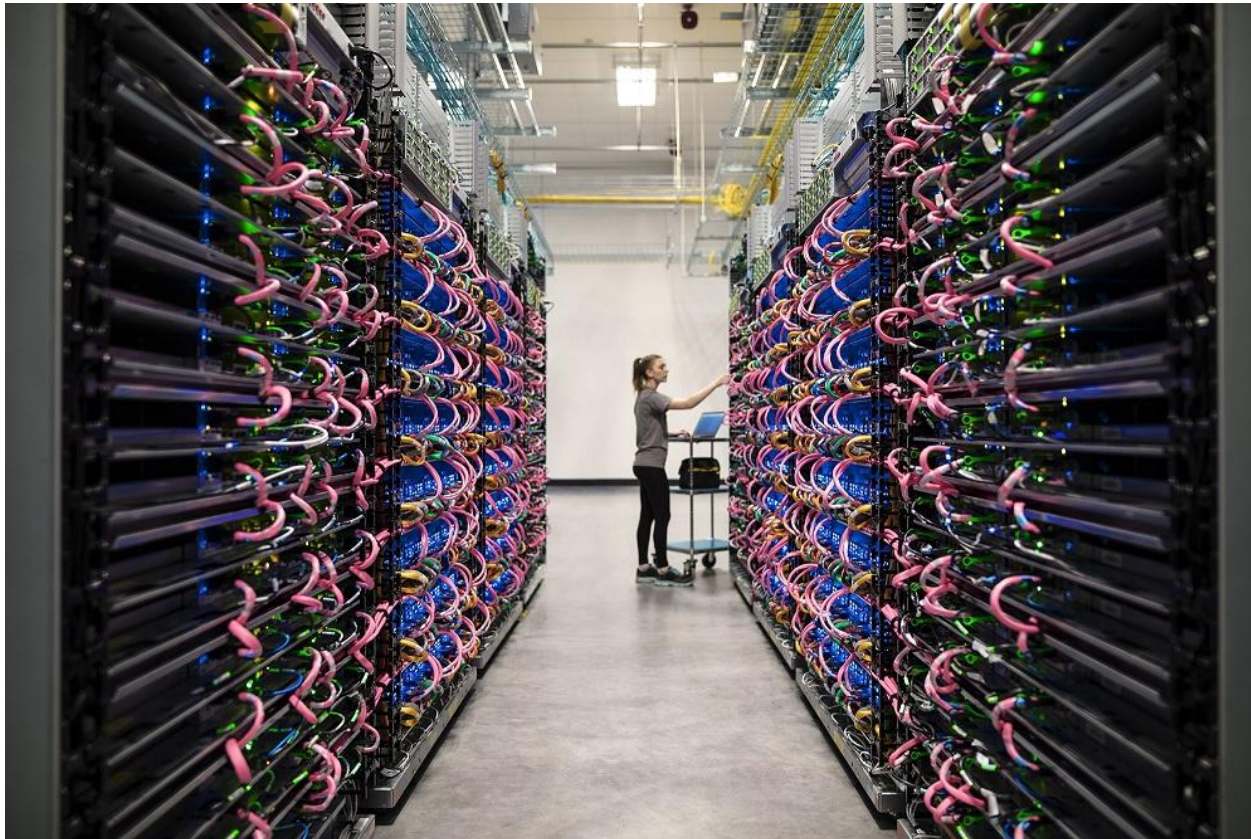
More information

- Bibliography, code, etc.
 - E.g. <https://deeplearning.net/> → opss ... closed after 2016
- Applications and theoretical foundations are continuously under developments
- AI/DL "increasingly being recognized as the latest GPT"
(GPT: General-purpose technologies: potential to drastically affect pre-existing economic and social structures)

And ... companies

- WHAT happens nowadays for the **hardware and company development** of Deep Learning (DL)?
- **Too fast** developments...
 - therein few examples just updated at the end of 2017
 - So it is now a sort of *historical view*
- Continuous improvements in recent years (everyday progress) and big *competition* among industries to ride the AI revolution

TPU @ Google



A server rack containing multiple **Tensor Processing Units**, a special-purpose chip designed specifically for machine learning

The original TPU was designed specifically to work best with Google's TensorFlow. Just for inference (mapping) !!!!

Nvidia

Keynote by Jensen Huang (Nvidia) movie:

https://www.youtube.com/watch?time_continue=407&v=F3g7A4q0sO8

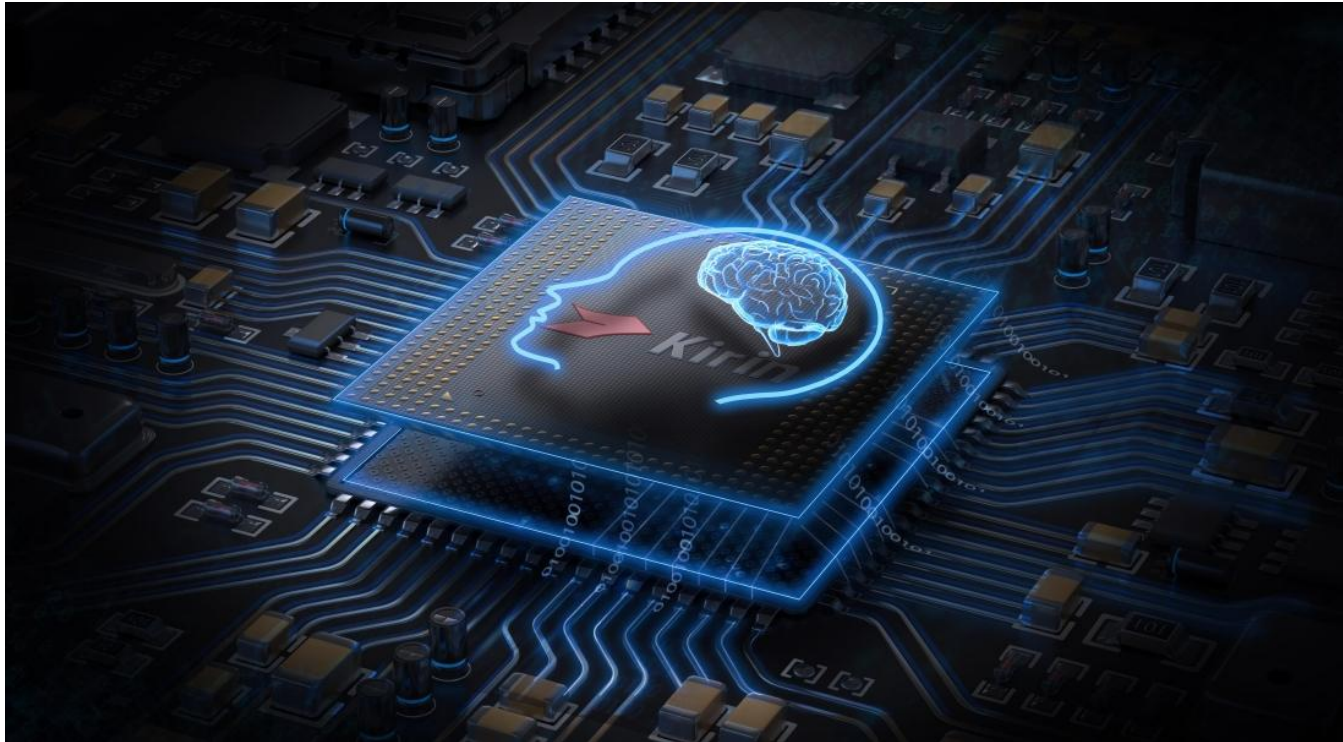
Changing Computer Science by new HW + DeepNN and recent applications

An example close to us: our Machine (Dept. of CS – Unipi):

c4130-p100 (Grant UniPi) October 2016, Dell PowerEdge with
4 Nvidia Tesla P100 16GB GPU, 512 GB RAM



NPU October 2017



Neural processing unit Huawei steps up its mobile chipset (!)
Kirin 970 is capable of processing 2000 images per second

Apple Neural Engine



- 2017: The Apple A11 Bionic (Iphone 8, X etc) includes dedicated neural network hardware that Apple calls a "Neural Engine". It can perform up to 600 billion operations per second and is used for Face ID, Animoji and other machine learning tasks.
- 2019: A13: octa-core neural engine for machine intelligence functions that can run over five trillion operations per second.
- Nowadays? ;-) **Check in your smartphone!**

Other architectures (guide slide)



Dip. Informatica
University of Pisa

Introduction to recent NN paradigms

- Other architectures.....
 - Deep learning
 - **Random weights NN**

AKA: *Deep* learning and *extreme* learning ;-)

Bibliography (NN third part)

Please follow the many references through the slides and the other Bibliography slides

Most of the main and basic parts are from the DL book:

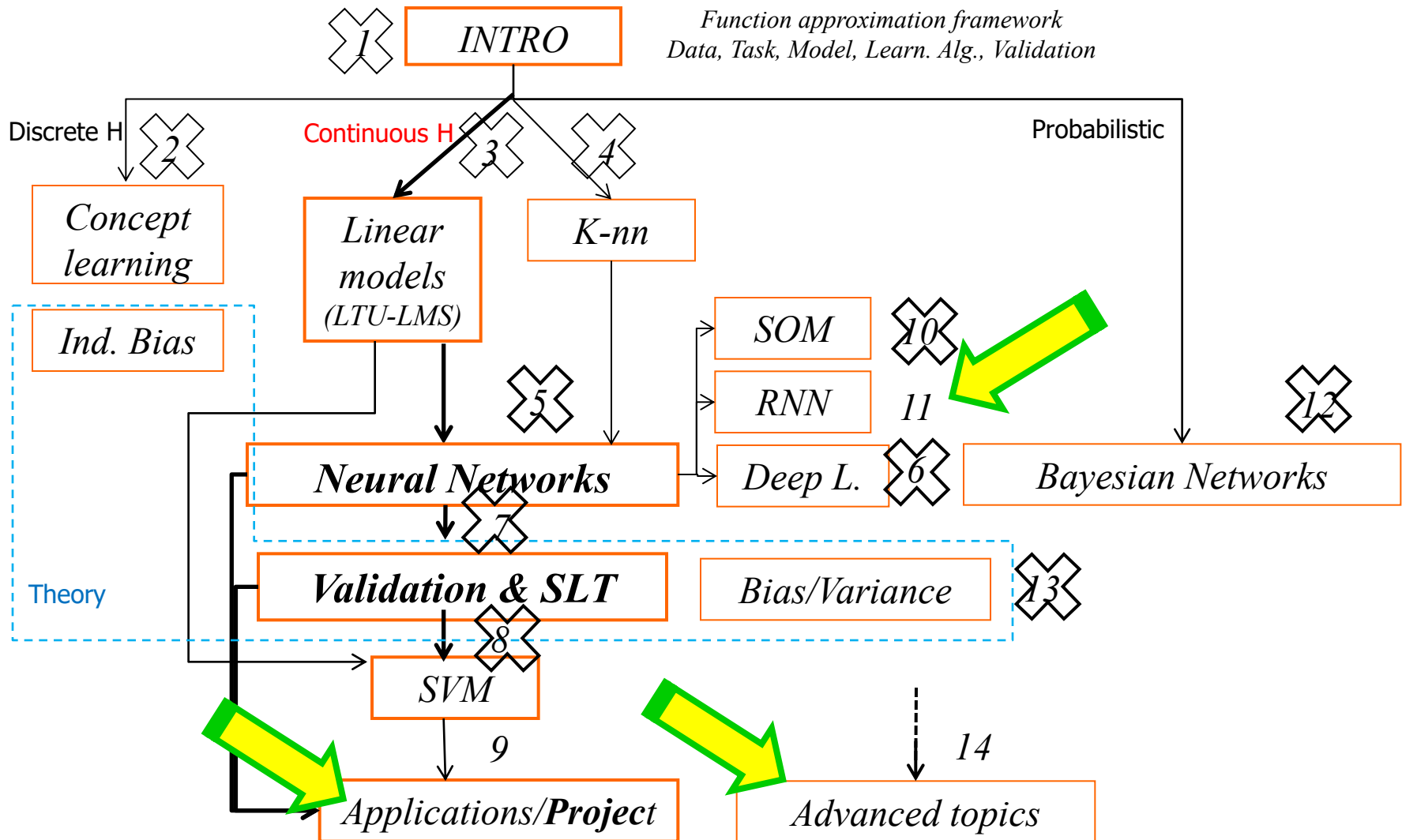
I. Goodfellow, Y. Bengio, A. Courville: ***Deep Learning***, MIT Press, 2016 . On-line: <http://www.deeplearningbook.org/>

ML Course structure

Where we go



Dip. Informatica
University of Pisa



Notes on Neural Networks: part 3

the end

Alessio Micheli

micheli@di.unipi.it

2025



Dipartimento di Informatica
Università di Pisa - Italy